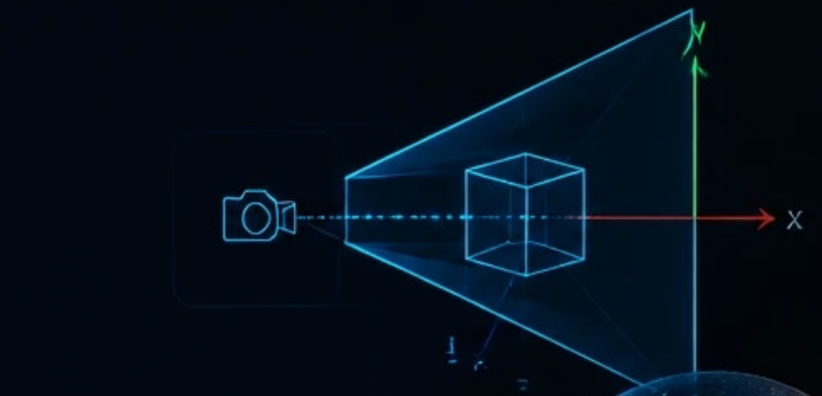
EF234504 — Grafika Komputer

Pertemuan 2

WebGL Fundamental & GLSL Basics



VERTEX SHADER

```
attribute vec3 position;
attribute vec3 normal;
uniform mat4 modelMatrix;
uniform mat4 viewMatrix;
uniform mat4 projectionMatrix;
varying vec3 vNormal;

void main() {
  vNormal = mat3(modelMatrix) * normal;
  gl_Position = projectionMatrix *
                viewMatrix *
                modelMatrix * vec4(position, 1.0);
}
```



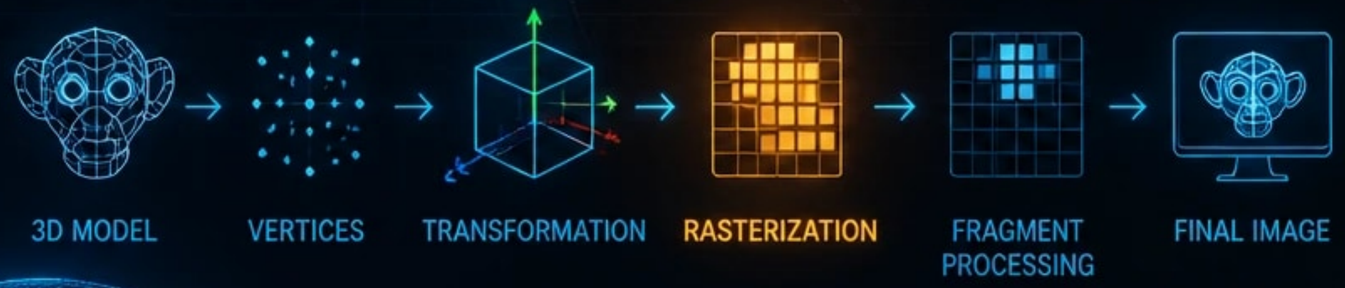
FRAGMENT SHADER

```
precision highp float;
varying vec3 vNormal;
uniform vec3 lightDir;
uniform vec3 baseColor;

void main() {
  vec3 N = normalize(vNormal);
  float diff = max(dot(N, -lightDir), 0.0);
  vec3 color = baseColor * (0.15 + 0.85 * diff);
  gl_FragColor = vec4(color, 1.0);
}
```

Lab: Grafika, Interaksi, Gim dan Analitik (GIGA)

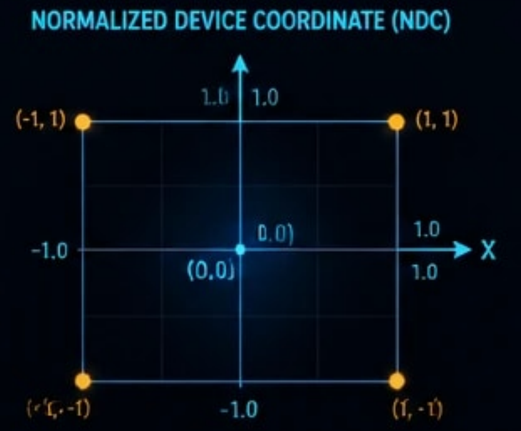
Departemen Teknik Informatika

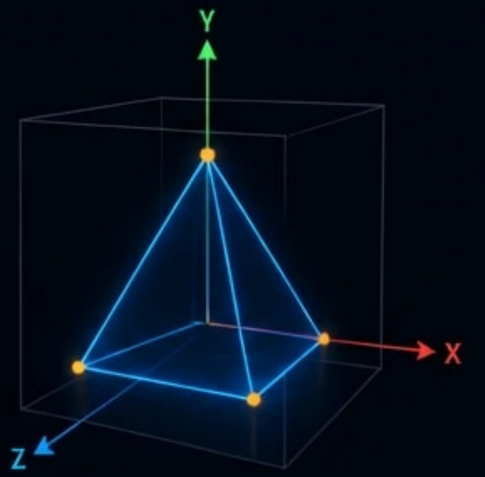


TOPIK PEMBAHASAN

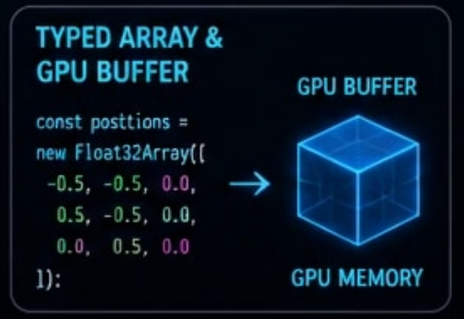
- Posisi WebGL dalam aplikasi grafika
- Canvas dan WebGL2 Context
- Normalized Device Coordinate
- Vertex dan Primitive
- Typed Array dan GPU Buffer
- Attribute
- GLSL dasar
- Vertex Shader
- Fragment Shader
- Uniform
- Data in / out
- Shader Program
- Draw Call
- Rendering Loop
- Praktikum: WebGL Primitive Playground







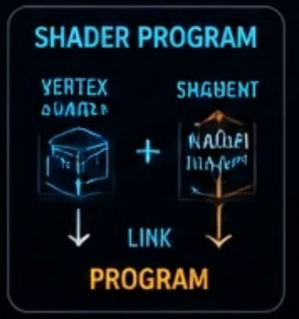


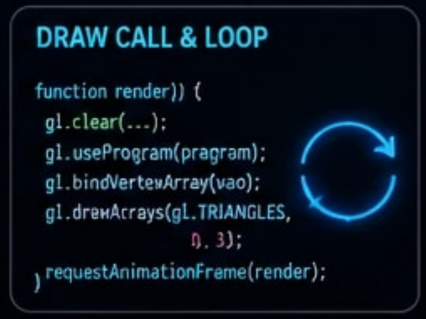












PRAKTIKUM: WEBGL PRIMITIVE PLAYGROUND Eksplorasi titik, garis, segitiga, dan berbagai primitive secara interaktif.

CAPAIAN PEMBELAJARAN

Setelah pertemuan ini mahasiswa mampu:

1. menjelaskan peran **WebGL** sebagai graphics API berbasis browser
2. membuat **WebGL2 Context**
3. menggunakan **NDC** untuk menentukan posisi
4. menjelaskan **vertex**, **primitive**, **buffer**, dan **attribute**
5. membaca struktur dasar **GLSL**
6. menjelaskan fungsi **Vertex Shader** dan **Fragment Shader**
7. membedakan **attribute**, **uniform**, dan **interpolated data**
8. melakukan **compile** dan **link** shader
9. melakukan **draw call**
10. membuat program **WebGL** sederhana dan interaktif

Apa Itu WebGL?

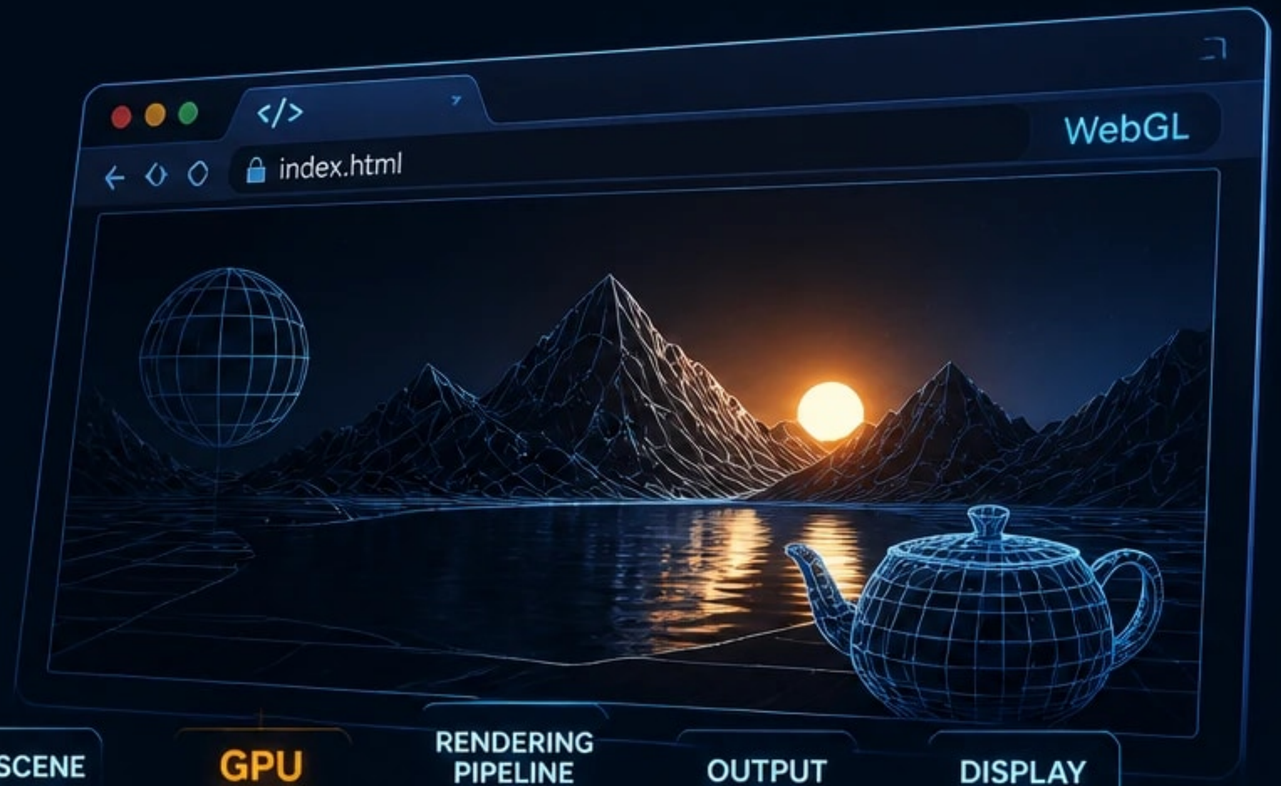
WebGL — Web Graphics Library

WebGL adalah API grafika pada browser yang memungkinkan **JavaScript** menggunakan **GPU**.

Digunakan untuk:

- grafika 2D/3D
- game web
- simulasi
- scientific visualization
- aplikasi interaktif

WebGL bersifat relatif **low-level**.





Grafika Komputer

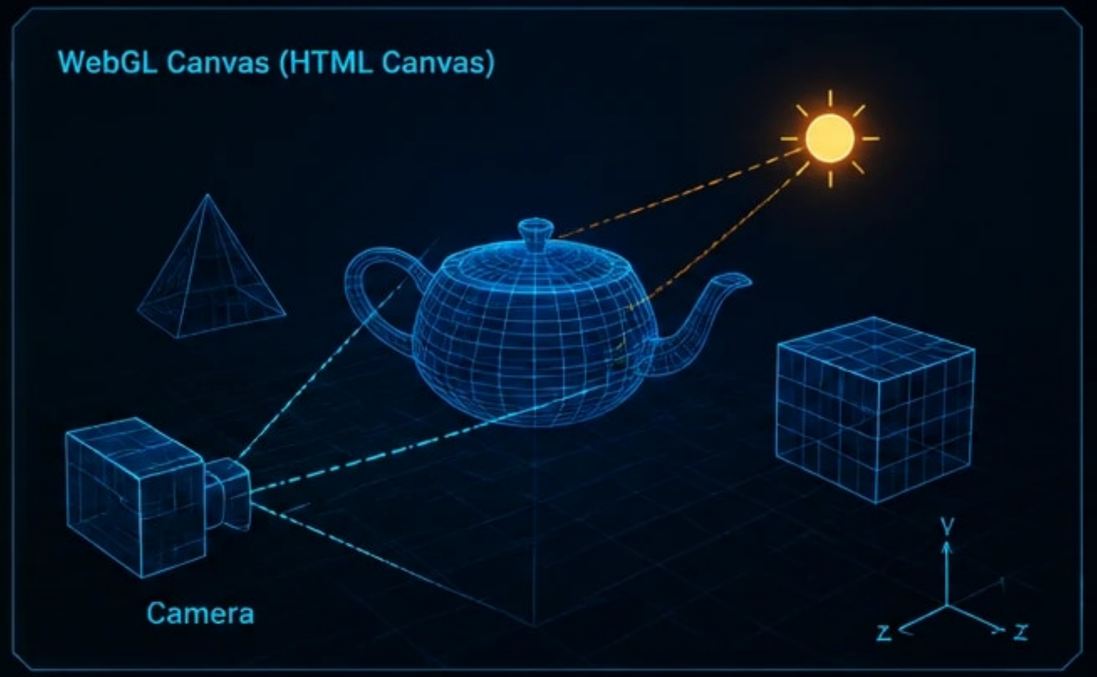
WEBGL BUKAN SCENE FRAMEWORK

WebGL TIDAK otomatis menyediakan:

- Scene
- Camera
- Mesh
- Light
- Material

WebGL menyediakan mekanisme dasar:

1. **Prepare Data**
   Siapkan vertex, index, uniform, dll.
2. **Upload to GPU**
   Kirim data ke GPU (buffers, textures)
3. **Run Shader**
   Vertex Shader → Fragment Shader
4. **Draw**
   GPU merasterize & menampilkan piksel



Framework seperti THREE.JS
akan membungkus proses ini.

Scene Graph · Camera · Mesh · Light · Material

Abstraksi & Manajemen Otomatis oleh Framework

WebGL = Low-level API untuk rendering grafis 3D di web.
Anda mengontrol semua: data, shader, pipeline, dan draw call.

Grafika Komputer

Posisi WebGL dalam Stack

WebGL menjadi **jembatan** antara **aplikasi JavaScript** dan **GPU**.





Application
JavaScript
WebGL API
Graphics Driver
GPU
Framebuffer
Canvas

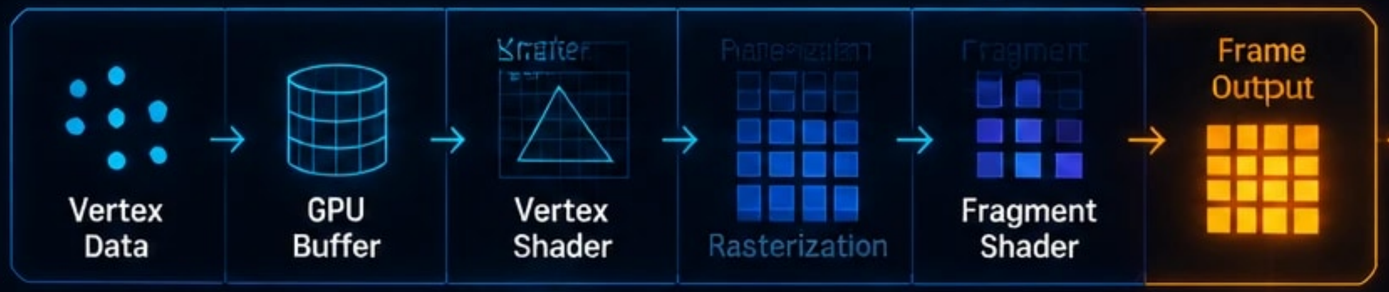
Struktur Program WebGL

01 Create Canvas
02 Get WebGL2 Context
03 Prepare Vertex Data
04 Create Buffer
05 Create & Compile Shader
06 Link Shader Program
07 Connect Attribute
08 Draw

> Ini adalah **workflow dasar** yang akan digunakan berulang kali.



HTML Canvas

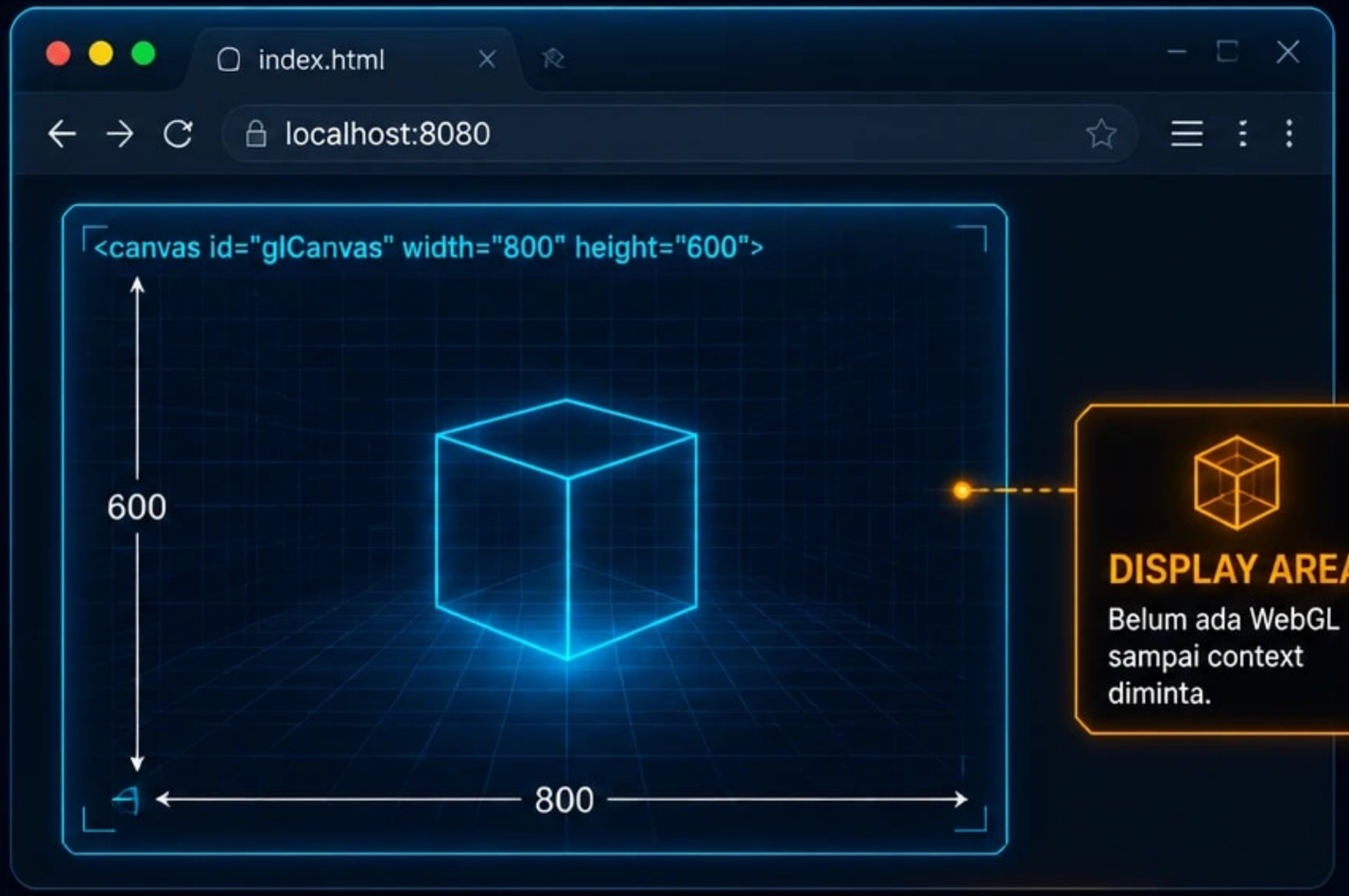




HTML Canvas

HTML

```
<canvas
  id="glCanvas"
  width="800"
  height="600"
</canvas>
```



Canvas adalah **area tampilan.**

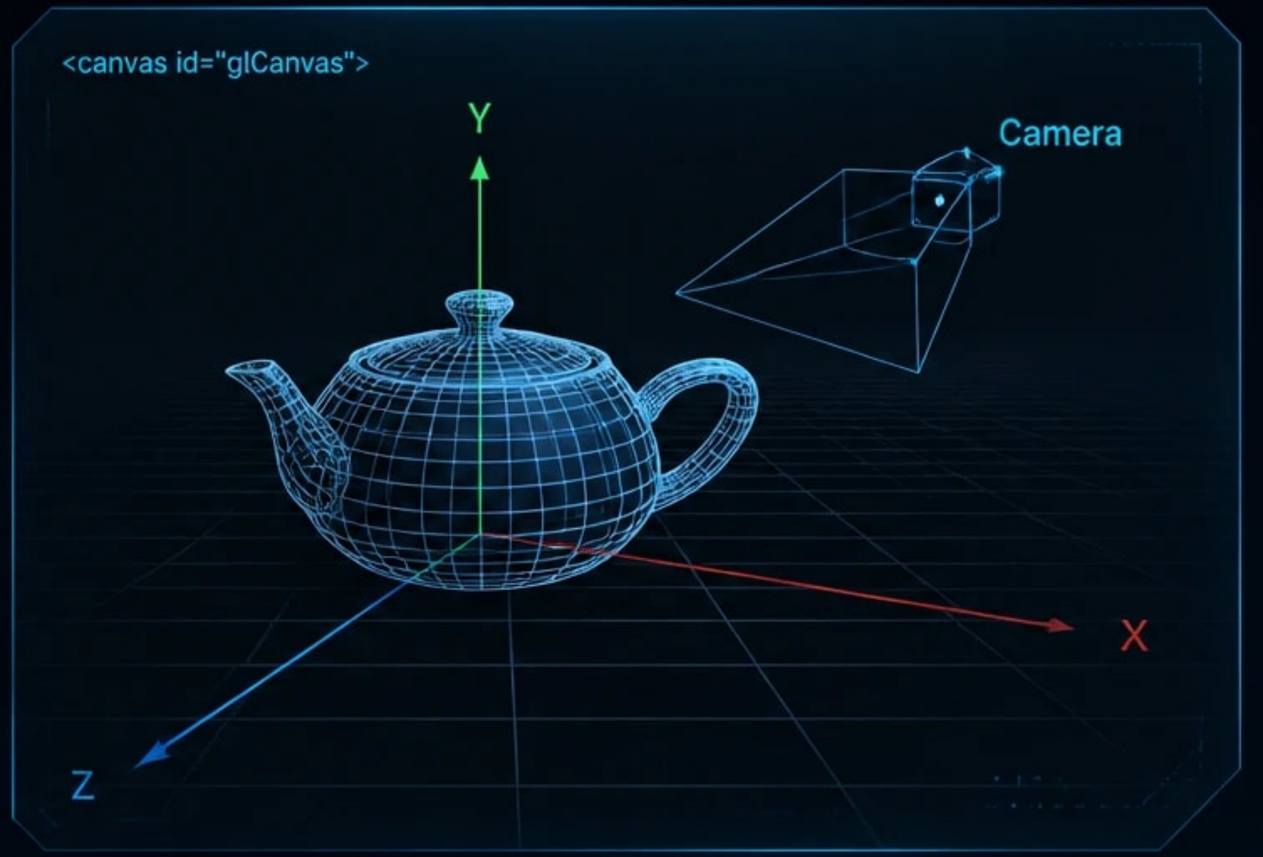
Canvas belum menjalankan **WebGL** sampai **JavaScript** meminta **WebGL Context**.

WebGL2 Context

```js
const canvas =
 document.getElementById("glCanvas");

const gl =
 canvas.getContext("webgl2");
```

`gl` adalah interface utama untuk memanggil operasi WebGL.



WebGL2 Context

- Modern 3D Graphics API
- WebGL 1.0 + Fitur Lanjutan
- Lebih Aman & Stabil
- Instanced Rendering
- Multiple Render Targets
- VAO, 3D Textures, dan banyak lagi

Memeriksa Context

```
if (!gl) {
    alert("WebGL2 tidak tersedia");
}
```

Context dapat gagal jika:

- browser tidak mendukung
- hardware acceleration tidak aktif
- GPU/driver bermasalah

Pastikan context valid sebelum membuat buffer, shader, atau melakukan draw call.

Grafika Komputer

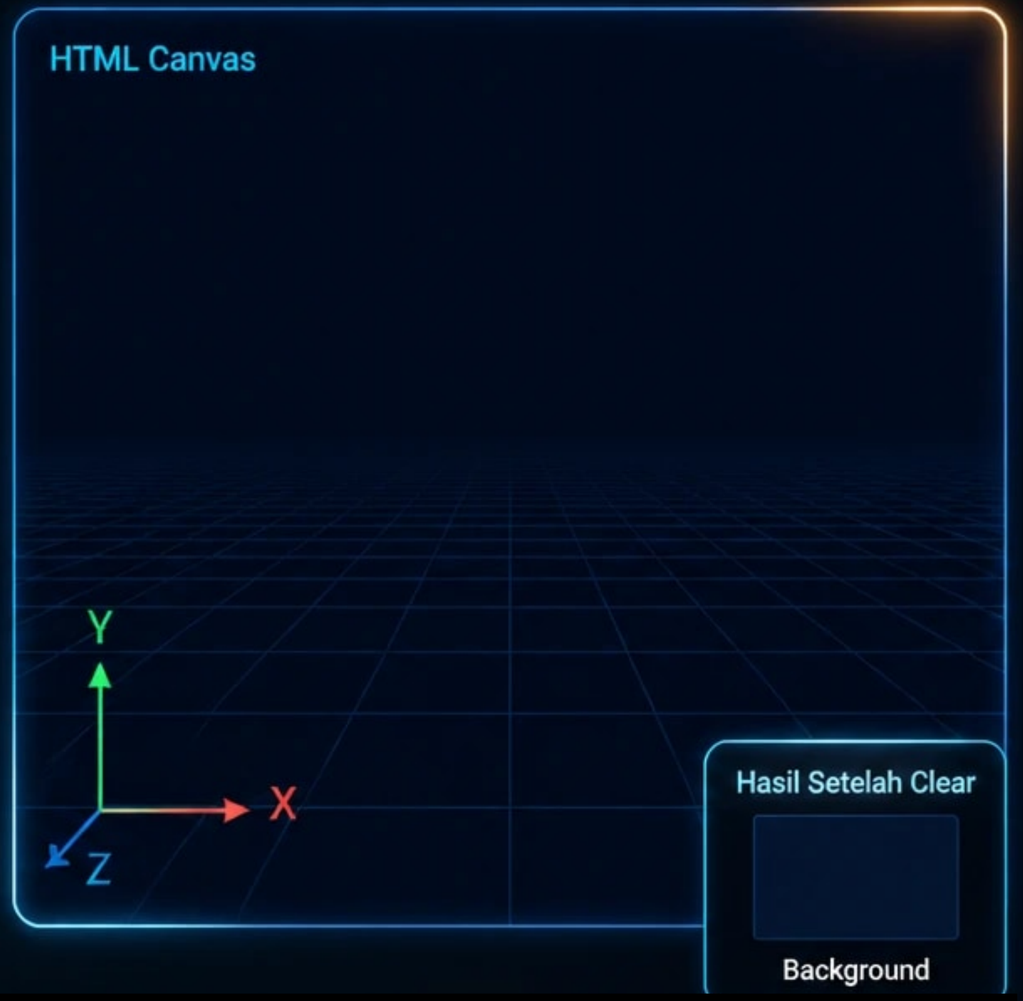
Membersihkan Canvas

```
gl.clearColor(
  0.05, 0.08, 0.15, 1.0
);
```

R	G	B	A
0.05	0.08	0.15	1.0

```
gl.clear(
  gl.COLOR_BUFFER_BIT
);
```

clearColor()
menentukan warna background framebuffer.



Warna pada WebGL

Nilai warna di WebGL

menggunakan rentang

0.0 – 1.0

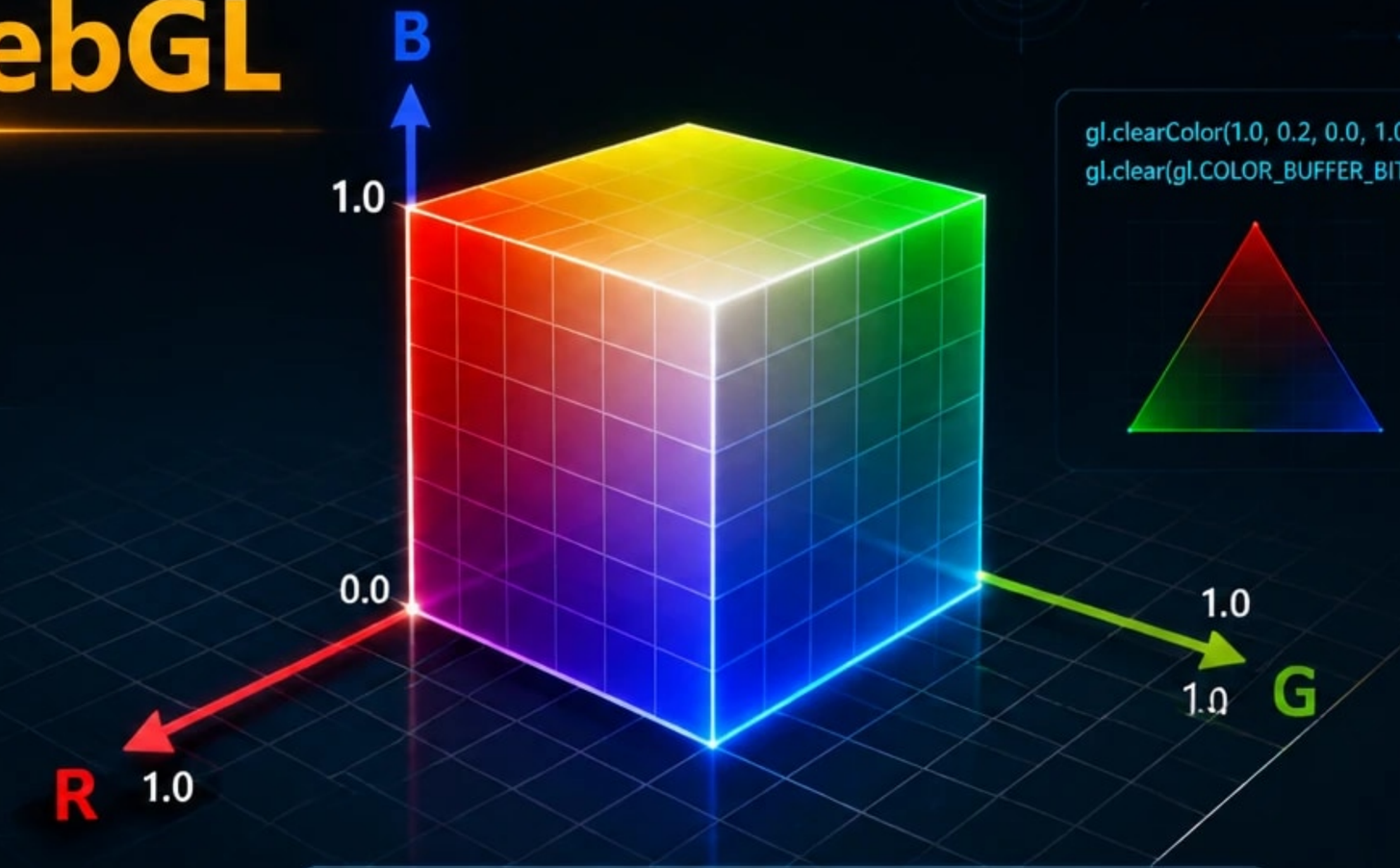
Contoh Warna Dasar

- Red = (1, 0, 0)
- Green = (0, 1, 0)
- Blue = (0, 0, 1)
- White = (1, 1, 1)

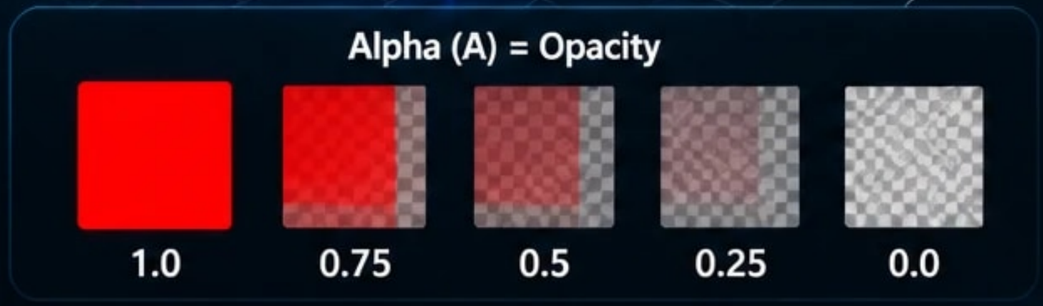
Format warna lengkap

RGBA

R = Red
G = Green
B = Blue
A = Alpha (Opacity)




```
gl.clearColor(1.0, 0.2, 0.0, 1.0);
gl.clear(gl.COLOR_BUFFER_BIT);
```

Canvas Coordinate vs NDC

CANVAS 2D (PIXEL COORDINATE)

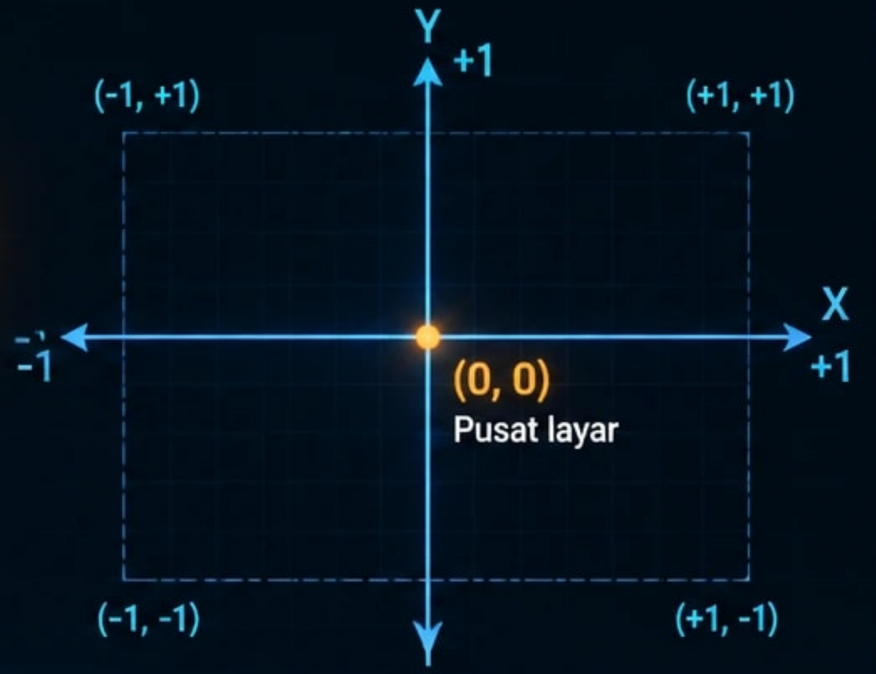
Canvas 2D menggunakan **koordinat pixel.**



VS

NORMALIZED DEVICE COORDINATE — NDC

WebGL menggunakan **Normalized Device Coordinate (NDC).**



RENTANG UMUM NDC

X : -1 → +1

Y : -1 → +1

Pusat layar:
(0, 0)

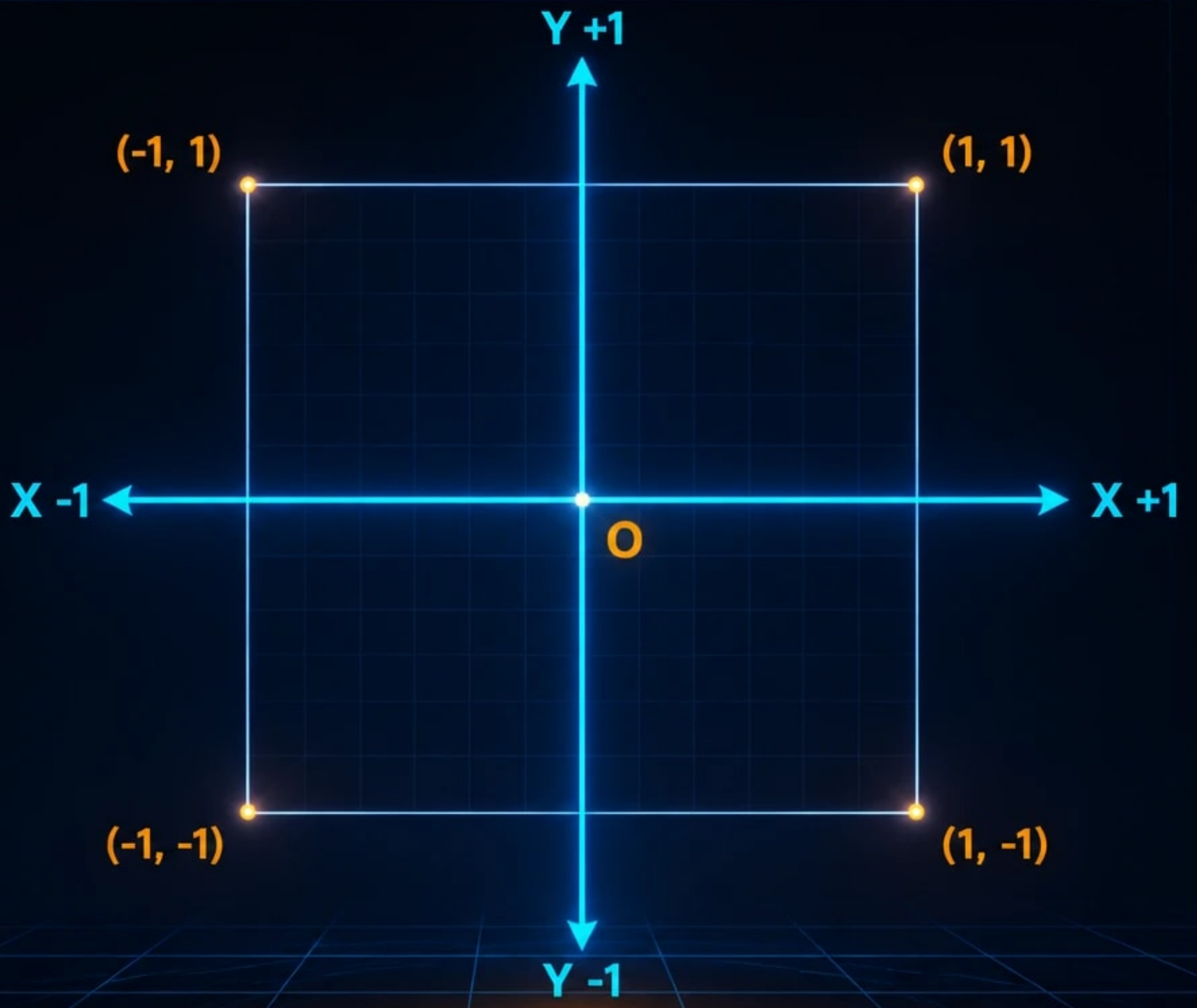
SISTEM KOORDINAT

NDC

Normalized Device Coordinate (NDC)
Ruang koordinat ter-normalisasi setelah proses transformasi dan projection.

POSISI PENTING

Kiri bawah	=	(-1, -1)
Kanan atas	=	(1, 1)
Tengah	=	(0, 0)



Mengapa NDC?

NDC membuat posisi **tidak bergantung** langsung pada ukuran **pixel Canvas**.

Render dapat bekerja **konsisten** di berbagai resolusi dan aspek rasio Canvas.

Konsep **viewport** dan koordinat layar akan muncul kembali pada materi transformasi dan **camera**.



Vertex

Vertex adalah unit data
yang diproses graphics pipeline.

Contoh Triangle

V0 = (-0.5, -0.5)
V1 = (0.5, -0.5)
V2 = (0.0, 0.5)



Setiap vertex dapat
membawa **data tambahan.**



Data per Vertex (Contoh)

Position	Normal	Color	UV	...
(x, y, z)	(nx, ny, nz)	(r, g, b, a)	(u, v)	

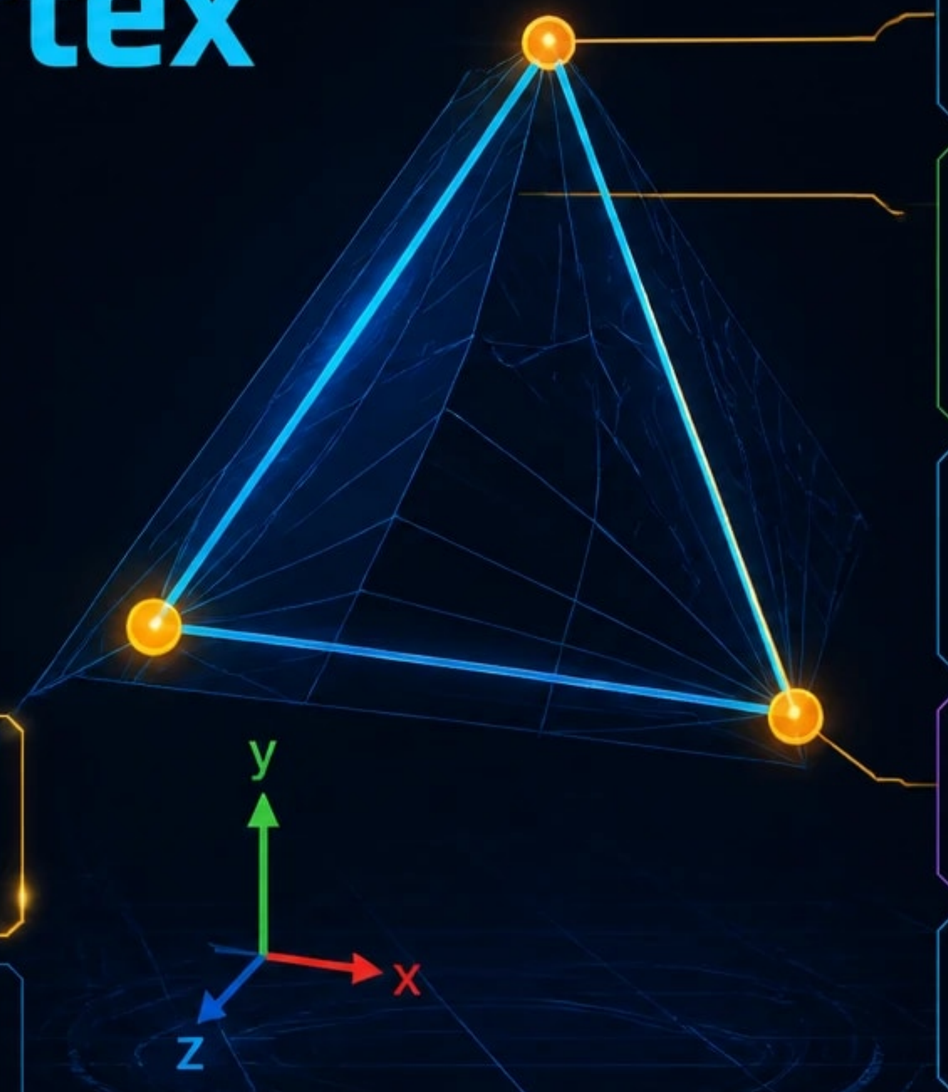
Data pada Vertex

- Position
- Color
- Normal
- Texture Coordinate
- Other Attributes

Pada pertemuan ini fokus:

Position + Color

Normal dan texture coordinate akan digunakan pada Pertemuan 5.



Position

x	0.38
y	0.72
z	0.15

Color

r	0.20
g	0.80
b	1.00
a	1.00

Normal

x	0.10
y	0.95
z	0.30

TexCoord

u	0.45
v	0.78

Other Attributes

... ...

Typed Array

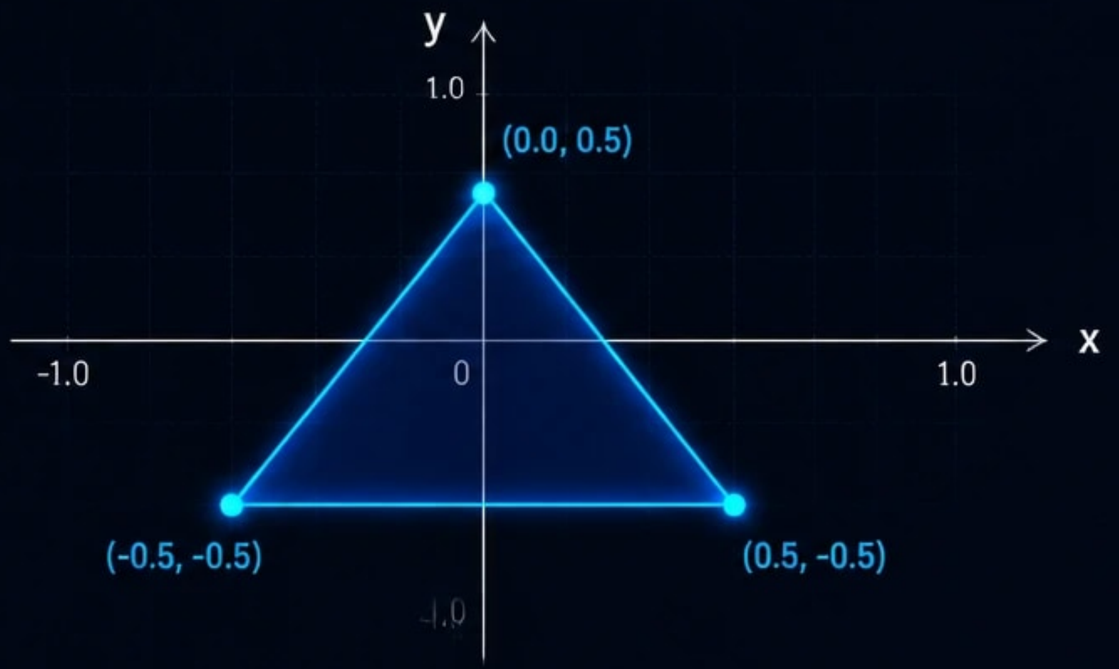
Contoh vertex data:

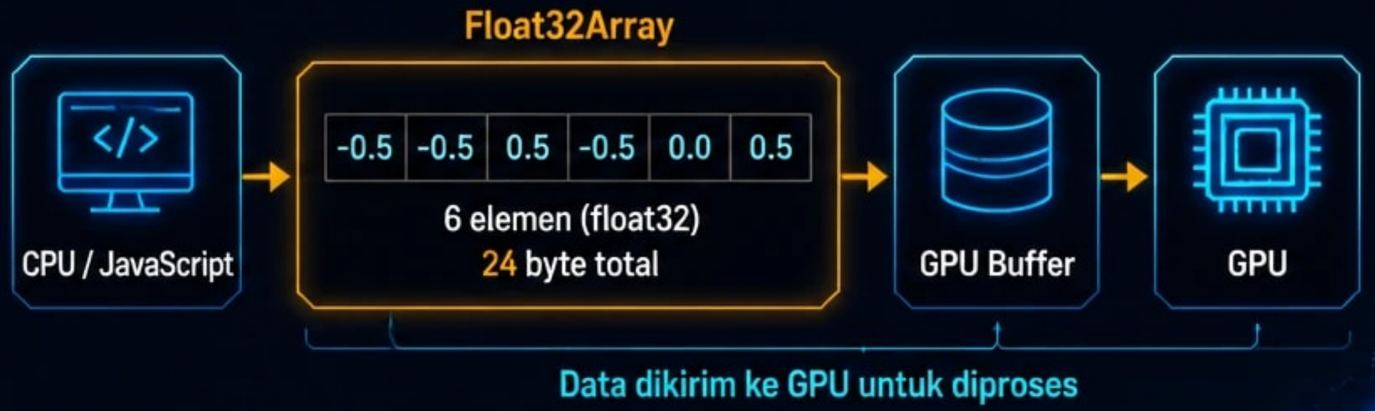
```
const vertices = new Float32Array([
    -0.5, -0.5,
     0.5, -0.5,
     0.0,  0.5
]);
```

Mengapa Float32Array?

- format data numerik terstruktur
- ukuran elemen konsisten
- cocok untuk data GPU

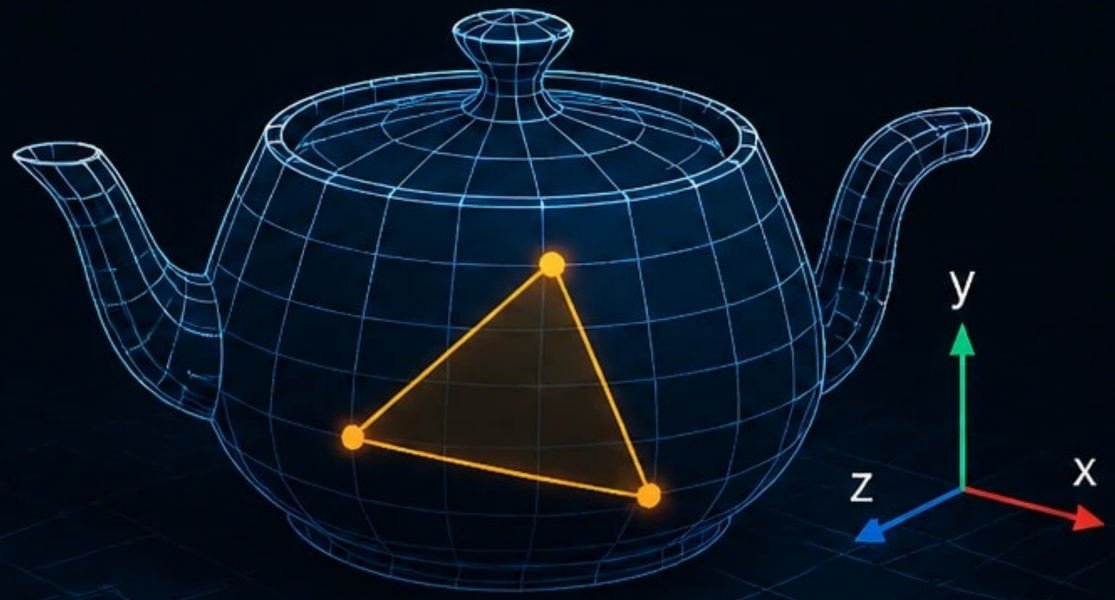
▷ 3 vertex (2D) membentuk 1 triangle





Primitive

Vertex dapat dirakit menjadi **primitive**.



PRIMITIVE UTAMA





Triangle adalah primitive terpenting untuk grafika 3D modern.

Draw Mode Primitive

Draw mode menentukan bagaimana urutan vertex ditafsirkan menjadi primitif geometri.



gl.POINTS



Setiap vertex digambar sebagai sebuah titik.



gl.LINES



Setiap pasangan vertex membentuk sebuah garis.



gl.LINE_STRIP



Setiap vertex (mulai dari ke-2) terhubung dengan vertex sebelumnya.



gl.LINE_LOOP



Seperti LINE_STRIP, namun vertex terakhir terhubung kembali ke vertex pertama.



gl.TRIANGLES



Setiap tiga vertex membentuk satu segitiga.



Urutan vertex + Draw Mode = Primitif Geometri

Satu data vertex yang sama dapat menghasilkan bentuk berbeda tergantung draw mode yang digunakan.

Primitive Assembly

Vertex Stream
urutan vertex dari buffer

Primitive Assembly
mengelompokkan vertex menjadi primitive

Point Line Triangle
Point / Line / Triangle

Untuk **gl.TRIANGLES**, setiap tiga vertex membentuk satu **triangle**.

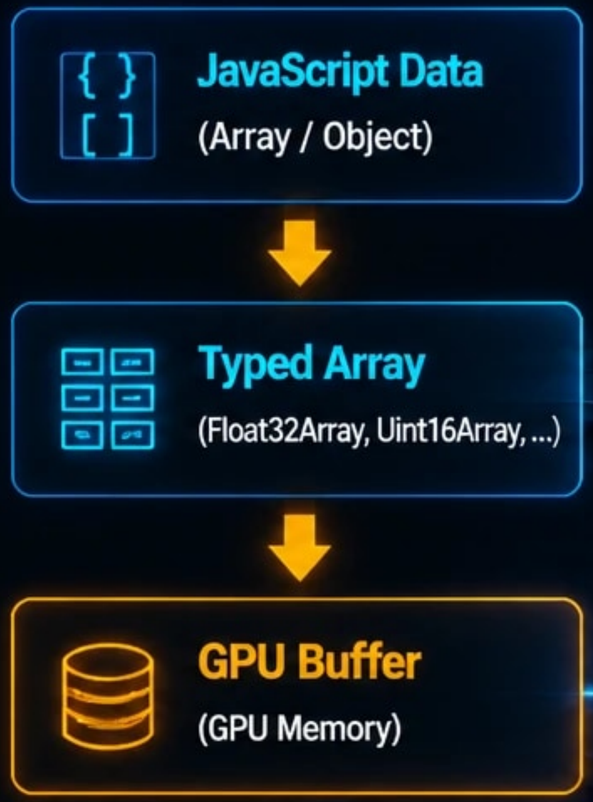


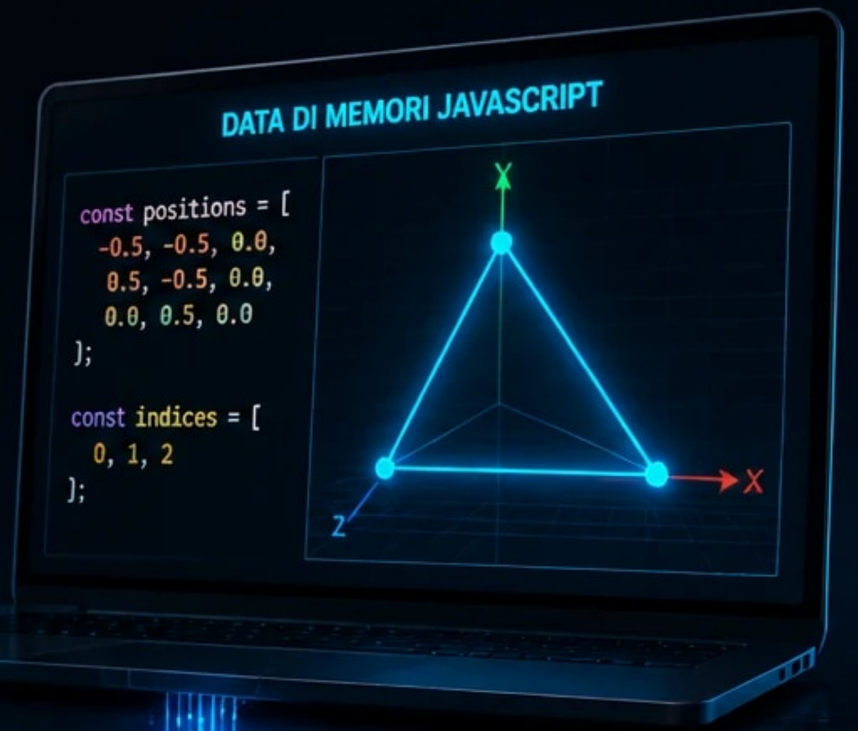
Mengapa Data Harus Masuk Buffer?

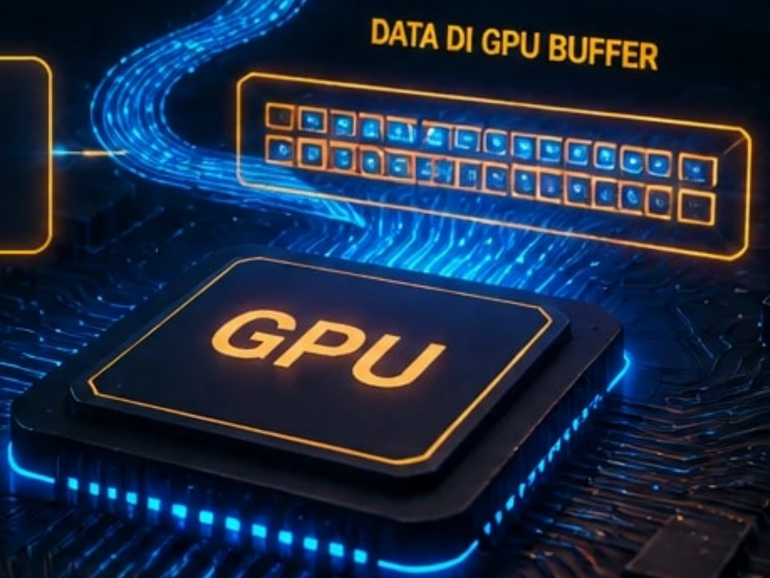
Data vertex awalnya berada di **memori JavaScript.**

Agar dapat dibaca GPU, data harus **dipindahkan** ke GPU Memory

Buffer adalah area **memori GPU** untuk data rendering.







Membuat dan Mengisi Buffer

Langkah dasar menyiapkan data vertex di GPU via WebGL

1 **Buat buffer**
Alokasikan objek buffer di WebGL

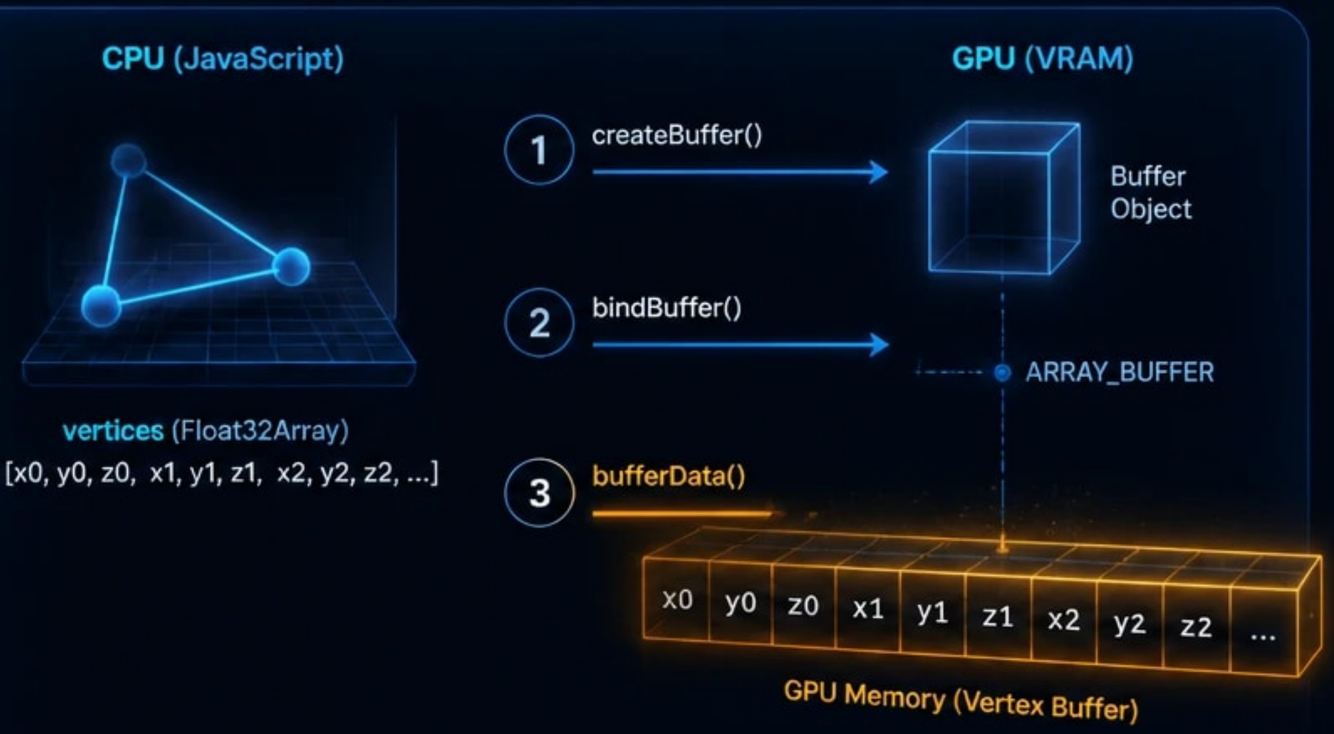
```
const positionBuffer =
  gl.createBuffer();
```

2 **Bind ke target**
Hubungkan buffer ke ARRAY_BUFFER

```
gl.bindBuffer(
  gl.ARRAY_BUFFER,
  positionBuffer
);
```

3 **Isi buffer**
Upload data vertex ke GPU memory

```
gl.bufferData(
  gl.ARRAY_BUFFER,
  vertices,
  gl.STATIC_DRAW
);
```



Alur Data



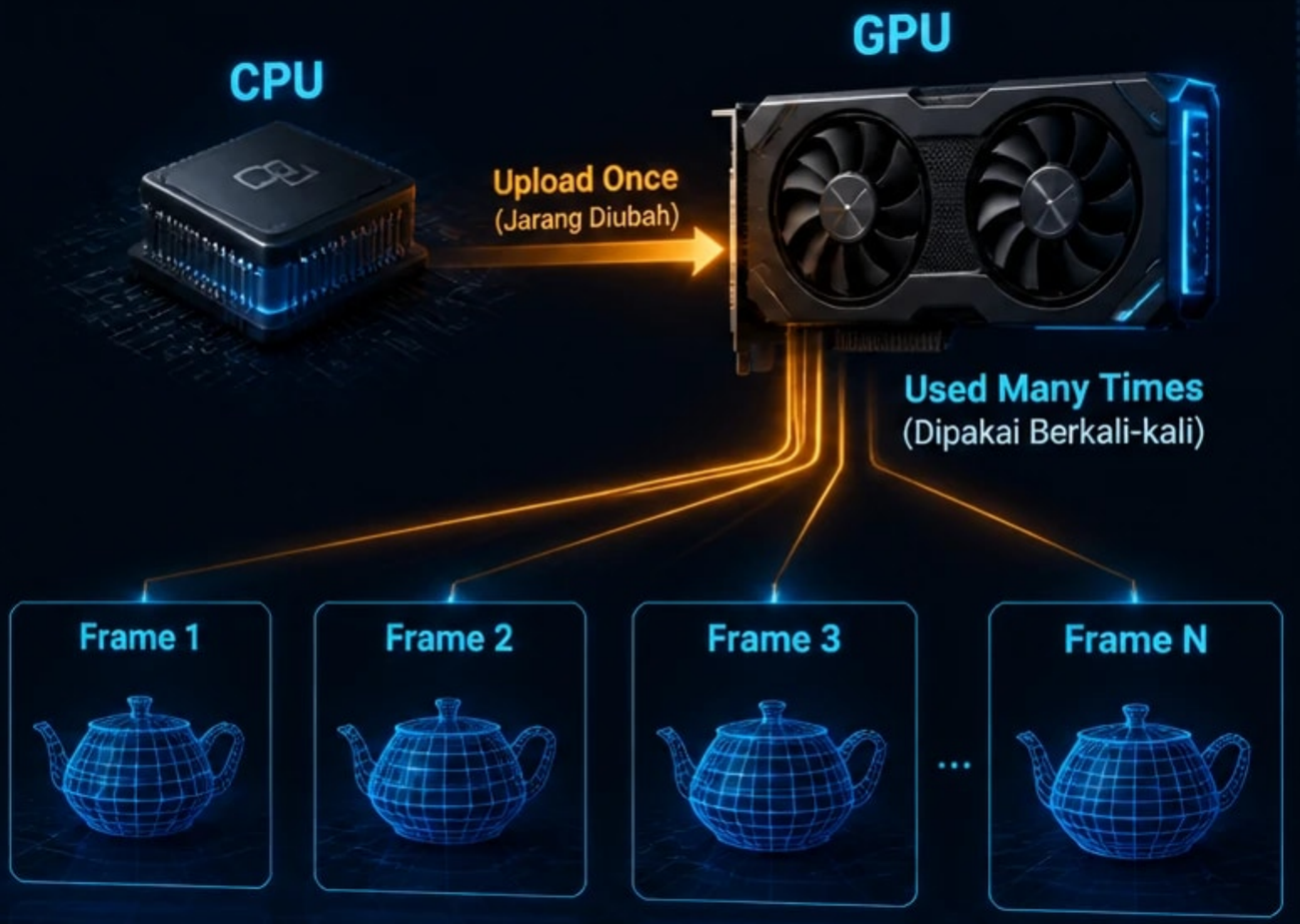
STATIC_DRAW

```
gl.STATIC_DRAW
```

merupakan **usage hint**.

Artinya secara umum:

- data relatif **jarang diubah**,
- **digunakan berkali-kali** saat rendering.



STATIC_DRAW memberi tahu WebGL bahwa data tidak sering berubah sehingga GPU dapat **mengoptimalkan penyimpanan dan performa**.

Attribute

Attribute adalah data yang berbeda untuk setiap vertex. (property)

Contoh (GLSL):

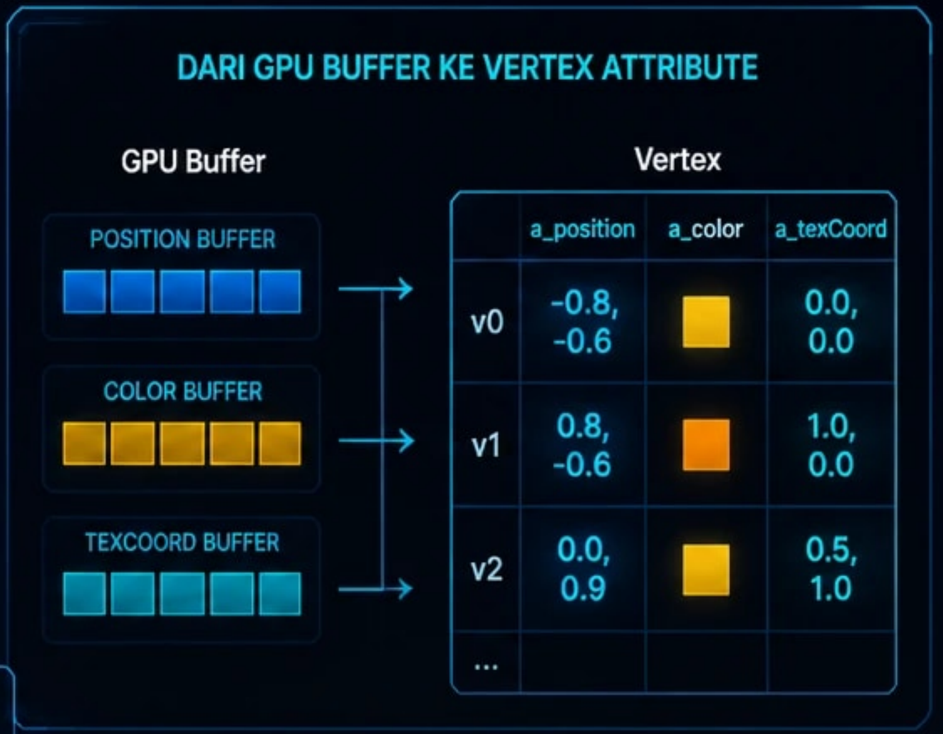
```
in vec2 a_position;
```

Attribute lain:

- color
- normal
- texture coordinate

Data attribute berasal dari **GPU buffer.**





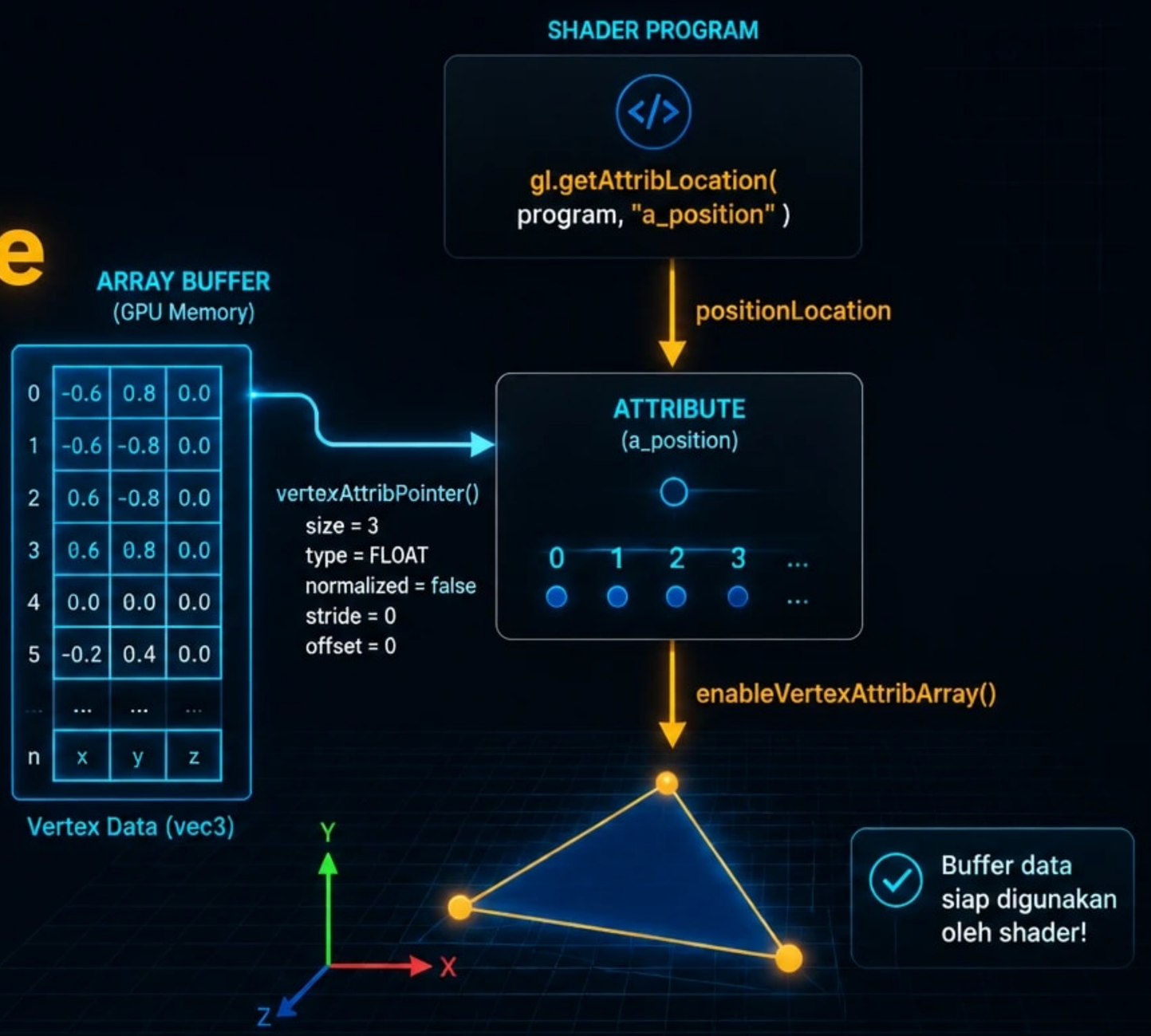


Menghubungkan Buffer ke Attribute

```
const positionLocation =
  gl.getAttribLocation(
    program,
    "a_position"
  );

gl.enableVertexAttribArray(
    positionLocation
  );
```

Kemudian layout ditentukan dengan `vertexAttribPointer()`.



Vertex Attribute Pointer

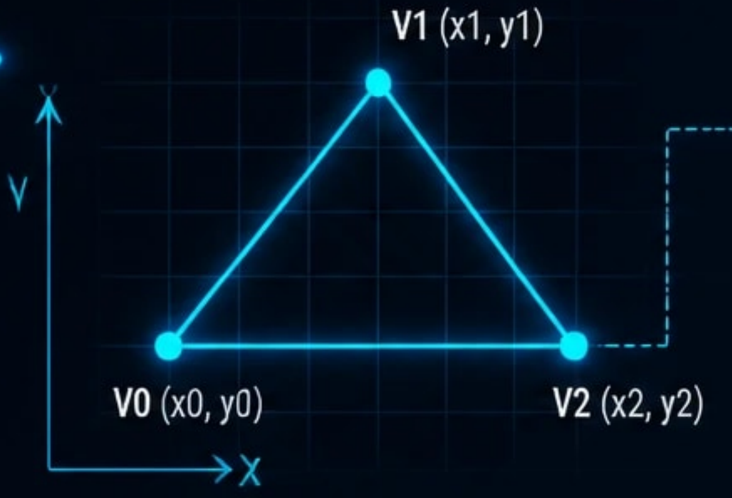
```
gl.vertexAttribPointer(
  positionLocation,   ——— attribute location
  2,                  ——— size (komponen per vertex)
  gl.FLOAT,           ——— tipe data
  false,              ——— normalisasi
  0,                  ——— stride (jarak antar vertex)
  0                   ——— offset (mulai dalam buffer)
);
```

2 komponen per vertex

→ X dan Y

Setiap vertex memiliki 2 nilai: X (horizontal) dan Y (vertical)

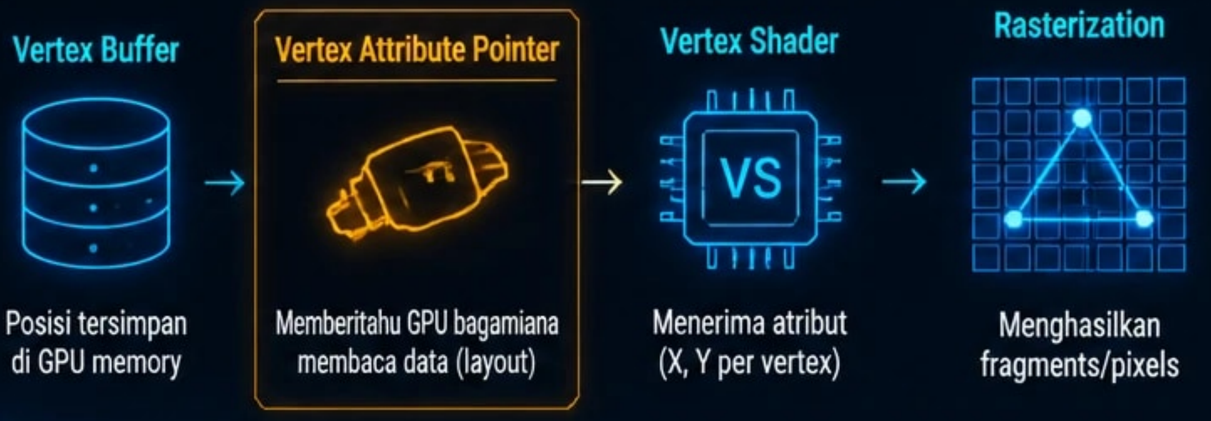
ATTRIBUTE: POSITION (vec2)



VERTEX BUFFER (POSITION)

0	x0	y0
1	x1	y1
2	x2	y2
	...	

DATA FLOW: ATTRIBUTE → VERTEX SHADER



Alur Buffer ke Shader

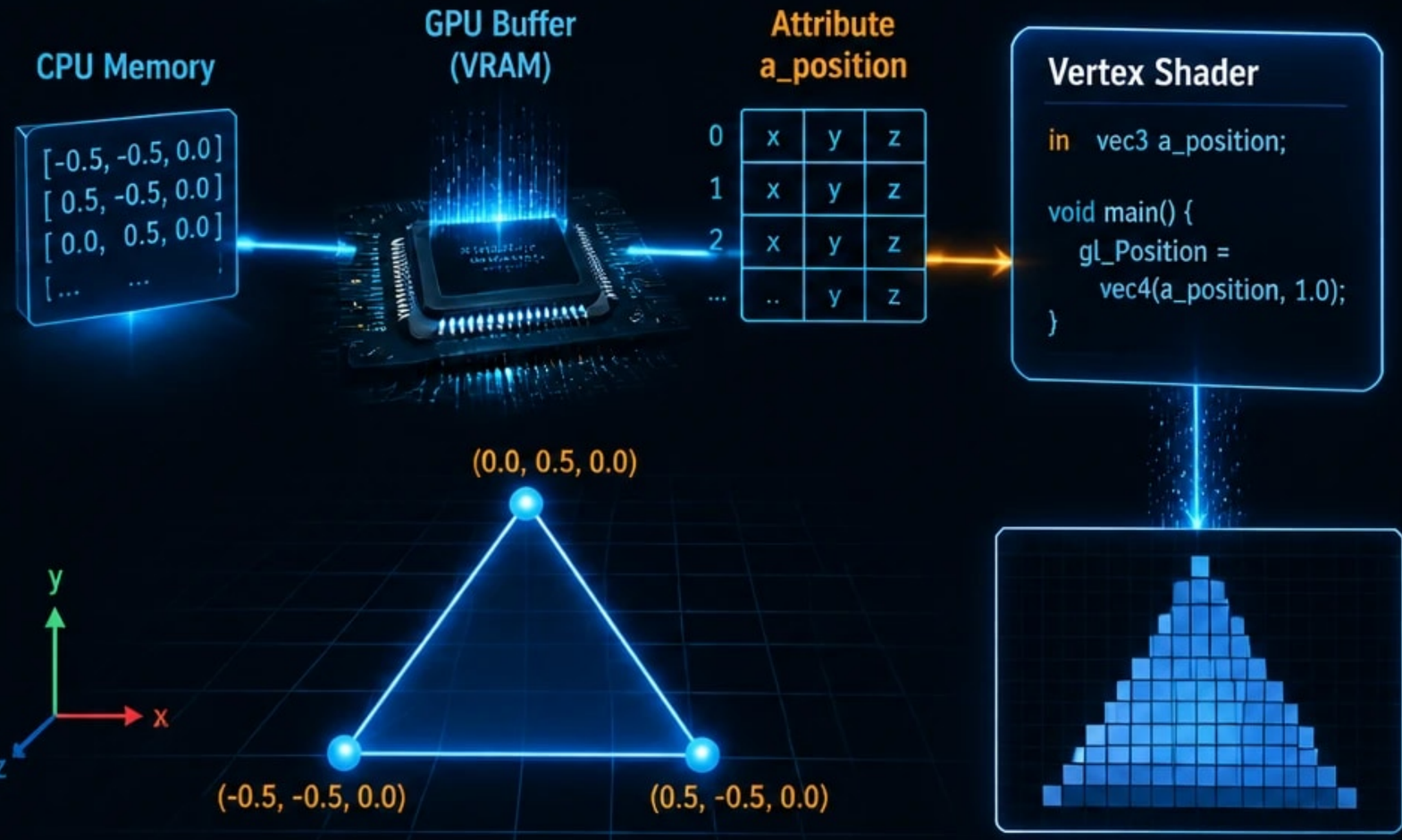
01 **Float32Array**
Data (CPU Side)

02 **GPU Buffer**
Data berada di VRAM

03 **vertexAttribPointer()**
Memberi format & lokasi atribut

04 **a_position**
Atribut position pada shader

05 **Vertex Shader**
Mengolah posisi vertex



Inti hubungan: Data (CPU) → Buffer (GPU) → Atribut → Vertex Shader → Posisi Vertex

Apa Itu GLSL?

GLSL — OpenGL Shading Language

GPU

GLSL digunakan untuk menulis program yang berjalan pada **GPU**.

WebGL2 menggunakan:

```
#version 300 es
```

1 0 0
0 1 0
0 0 1

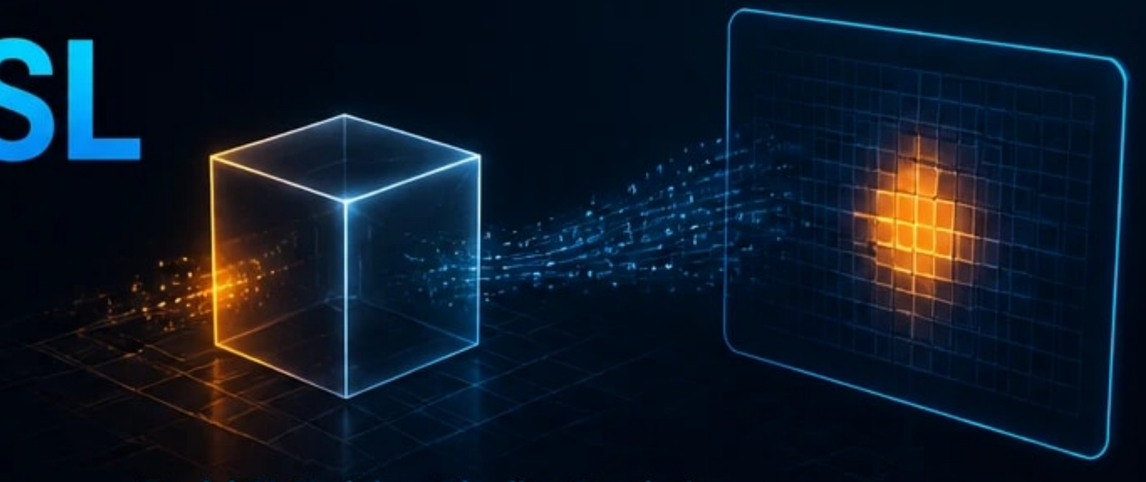
GLSL dirancang untuk operasi grafika dan **vektor/matriks.**



</>

GLSL memungkinkan **kontrol penuh** atas bagaimana objek 3D diproses, dirasterisasi, dan diberi warna **di tingkat GPU**.

Struktur Dasar GLSL



GLSL SHADER

Pola dasar setiap shader

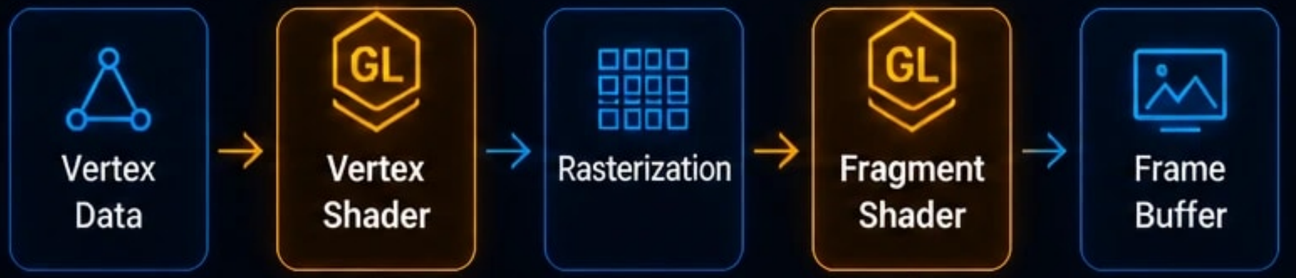
```
1  #version 300 es
2
3  void main() {
4      // shader code
5  }
```

PRECISION

Fragment Shader membutuhkan precision declaration

```
1  precision highp float;
2
```

Posisi GLSL dalam Pipeline Rendering



Diproses **per-vertex**
Transformasi posisi, normal, color, dll.

Diproses **per-fragment**
Warna, lighting, texturing, alpha, dll.

GLSL adalah bahasa shading untuk mengontrol bagaminana vertex dan fragment diproses di GPU.

Tipe Data Dasar GLSL



SCALAR
Satu nilai

- f — float
- i — int
- b — bool

VECTOR
Kumpulan komponen bertipe sama

- vec2
- vec3
- vec4

MATRIX
Transformasi linear

- mat2
- mat3
- mat4

SAMPLER
Mengambil nilai dari texture

sampler2D





Tipe data GLSL digunakan untuk menyimpan nilai, melakukan perhitungan, dan mengirimkan data antar shader.

Vector dan Matrix pada GLSL

Contoh Vector

```
vec2   position2D;
vec3   position3D;
vec4   color;
```

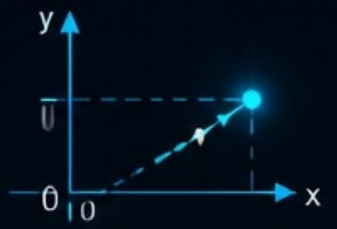
Contoh Matrix

```
mat4   transform;
```

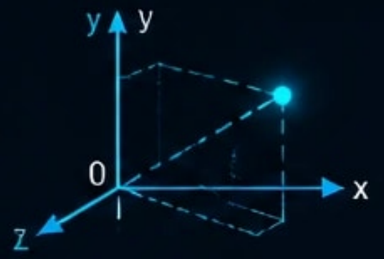
Vector digunakan untuk **posisi**, **arah**, warna, dan **UV**.

Matrix banyak digunakan untuk **transformation**.

vec2
(x, y)



vec3
(x, y, z)



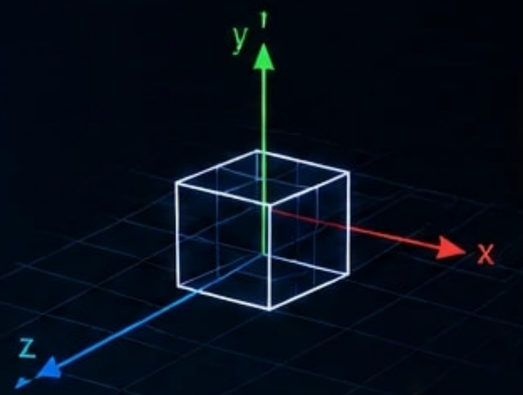
vec4
(r, g, b, a)



mat4

$$\begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

Sebelum Transformasi

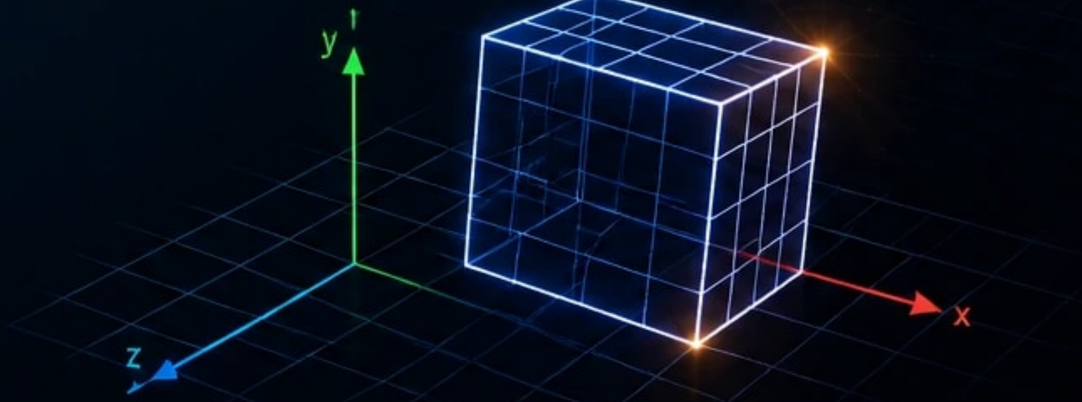


Transform Matrix (mat4)

1.2	0.0	0.0	2.0
0.0	1.2	0.0	1.0
0.0	0.0	1.2	3.0
0.0	0.0	0.0	1.0

Transformasi

Sesudah Transformasi

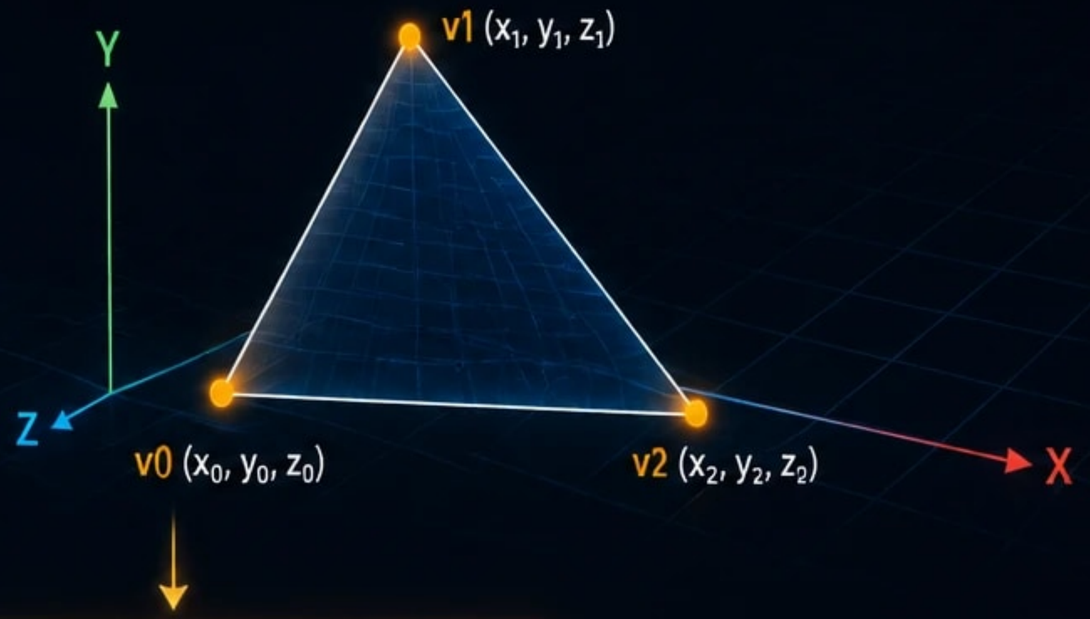


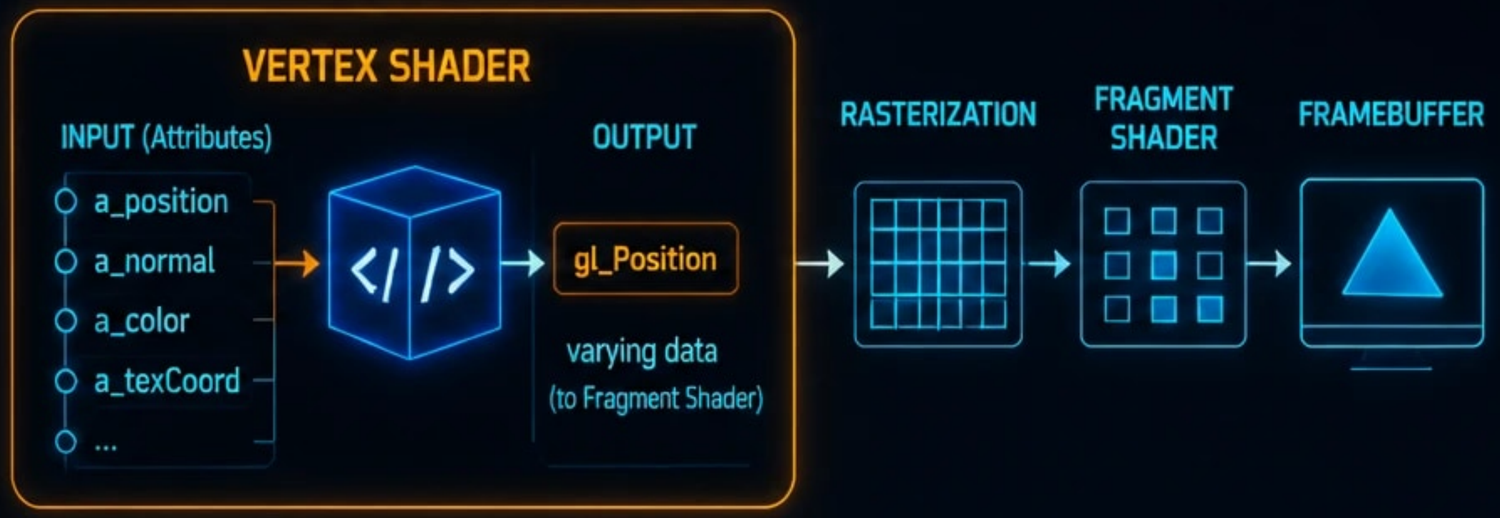
Vertex Shader

Vertex Shader berjalan:
satu kali untuk setiap **vertex**.

Tugas dasar:

- menerima **attribute**,
- memproses **posisi**,
- menghasilkan **gl_Position**,
- mengirim data ke **tahap berikutnya**.





Grafika Komputer

Vertex Shader Sederhana

GLSL ES 3.00 </>

```
#version 300 es

in vec2 a_position;

void main() {
   gl_Position =
     vec4(
        a_position,
        0.0,
        1.0
       );
}
```

Input atribut: posisi vertex 2D

gl_Position: output posisi vertex (clip space)

gl_Position
adalah output posisi vertex.

FRAGMENT SHADER

Fragment Shader berjalan untuk fragment yang dihasilkan rasterization.

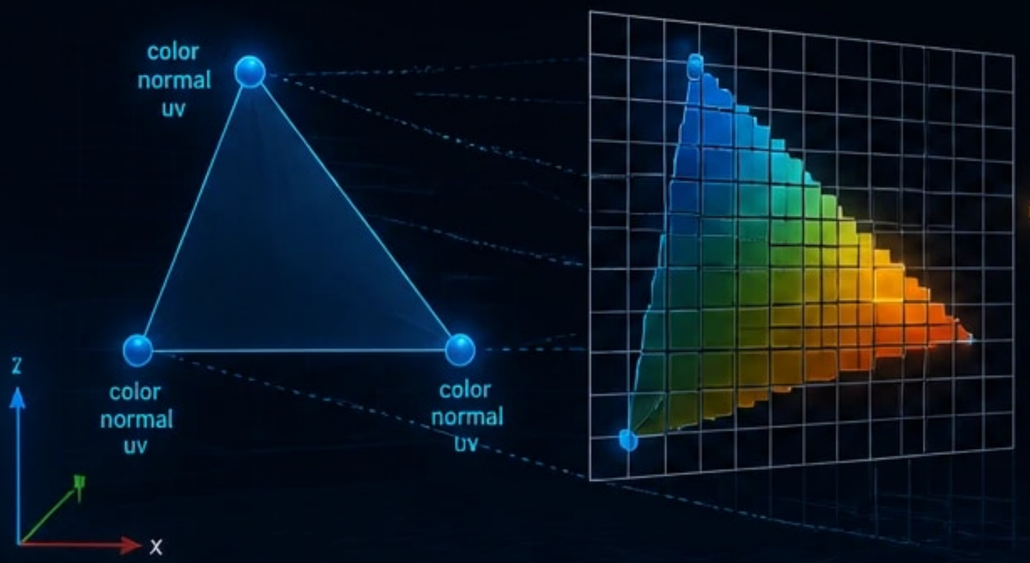
TUGAS DASAR

MENENTUKAN WARNA
Menghasilkan warna akhir (RGBA) untuk setiap fragment / pixel.

MENERIMA INTERPOLATED DATA
Menerima data dari Vertex Shader yang sudah di-interpolasi.

MENGGUNAKAN TEXTURE & LIGHTING
Dapat memakai texture, normal map, dan pencahayaan untuk hasil yang realistis.



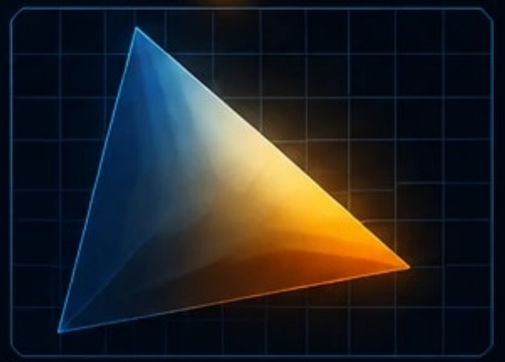


FRAGMENT SHADER

Untuk setiap fragment:

- ✓ menerima interpolated data (color, normal, uv, dll)
- ✓ bisa sampling texture
- ✓ bisa hitung pencahayaan (lighting)

Output: warna (RGBA)



TEXTURE
Sampling texture menggunakan UV

NORMAL MAP
Menggunakan normal untuk lighting detail

LIGHTING
Hitung pencahayaan per fragment

Grafika Komputer —

Fragment Shader Sederhana

Menghasilkan warna akhir pada setiap *fragment* (piksel) yang lolos dari tahap rasterization.

Output: **vec4 (RGBA)**
Warna keluaran untuk piksel.

Pada contoh ini,
setiap piksel diwarnai
biru muda solid.

GLSL ES 3.00
WebGL2 Fragment Shader

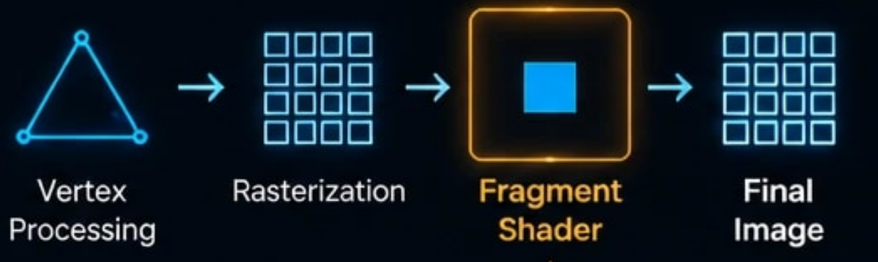
GLSL ES 3.00

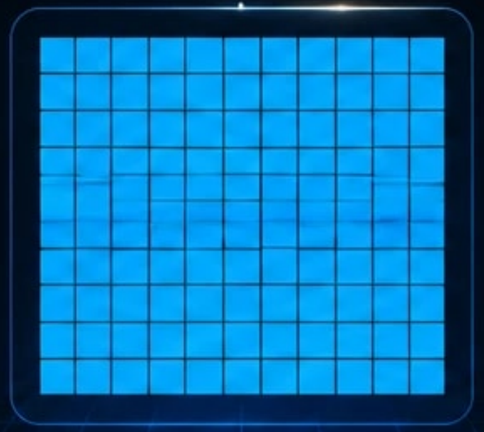
```
#version 300 es
precision highp float;

out vec4 outColor;

void main() {
    outColor =
        vec4(0.0, 0.7, 1.0, 1.0);

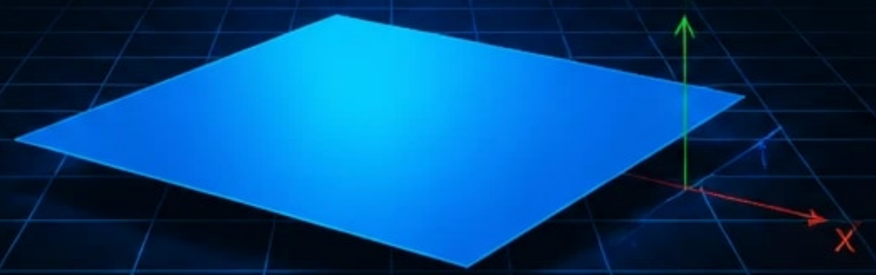
}
```





Output Warna

RGBA
(0.0, 0.7, 1.0, 1.0)



Fragment Shader menentukan **warna akhir setiap piksel** berdasarkan data interpolated dan uniform.

Vertex Shader vs Fragment Shader

VERTEX SHADER
per vertex

- per vertex
- mengolah posisi
- menerima vertex input
- menghasilkan gl_Position



FRAGMENT SHADER
per fragment

- per fragment
- mengolah warna
- menerima interpolated input
- menghasilkan final fragment color



Konsep ini tidak akan dijelaskan ulang pada P3–P5; hanya digunakan.

Uniform

Uniform adalah data yang **sama** untuk **satu draw call**.

Contoh:

```
uniform float u_time;
uniform vec4 u_color;
uniform mat4 u_matrix;
```

Perbedaan Penting

Attribute	→	per vertex
Uniform	→	per draw call
out/in	→	antar-stage shader

Uniform Data Flow (per draw call)



Sebutkan variabel apa yang bisa kita buat uniform?

Uniform dikirim ke GPU **sekali saja** untuk satu **draw call** lalu digunakan oleh **semua vertex** dan **fragment**.

Rasterization

Triangle

↓

Rasterization

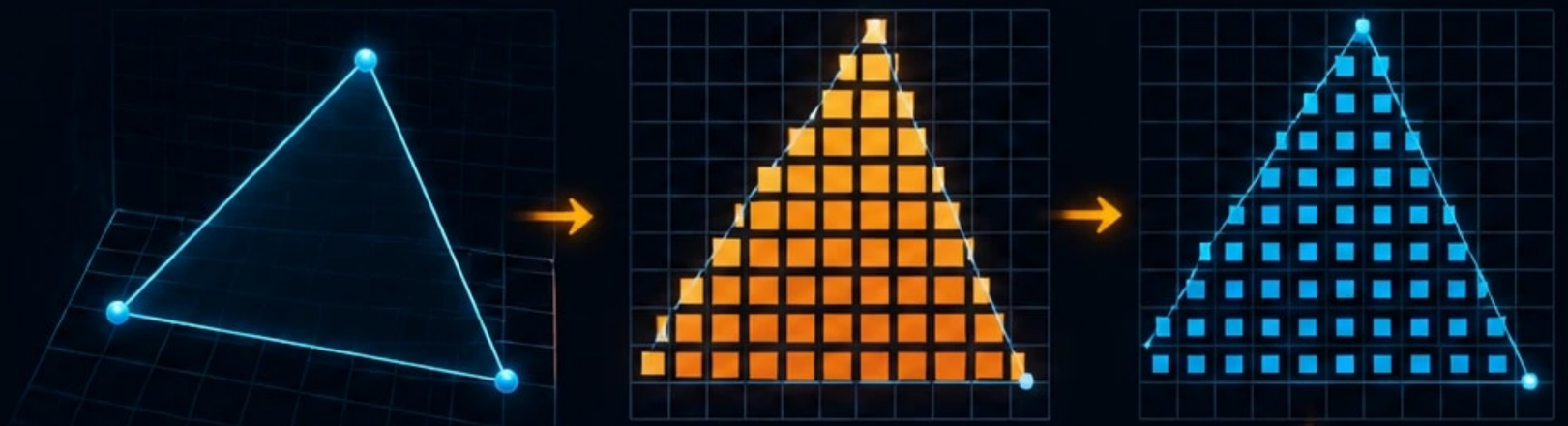
↓

Fragments

↓

Fragment Shader

TRIANGLE → RASTERIZATION → FRAGMENTS



FRAGMENT SHADER

</>

Menghitung warna & kedalaman setiap fragment

Satu triangle dapat menghasilkan **banyak fragment.**



Compile dan Link Shader

Proses membangun program shader dari source GLSL hingga siap digunakan.

Kesalahan syntax dapat diperiksa melalui shader/program **info log**.

GLSL Source

↓

Compile Vertex Shader

↓

Compile Fragment Shader

↓

Link

↓

Shader Program

Vertex Shader (GLSL)

```
#version 300 es
in vec3 aPosition;
in vec3 aColor;
uniform mat4 uMVP;
out vec3 vColor;

void main() {
  vColor = aColor;
  gl_Position = uMVP *
    vec4(aPosition, 1.0);
}
```

Fragment Shader (GLSL)

```
#version 300 es
precision mediump float;
in vec3 vColor;
out vec4 fragColor;
void main() {
  fragColor = vec4(vColor, 1.0);
}
```



Grafika Komputer

Shader Program

Vertex Shader dan **Fragment Shader** harus dihubungkan menjadi satu **program**.



Digunakan dengan:

```
gl.useProgram(program);
```

Draw Call

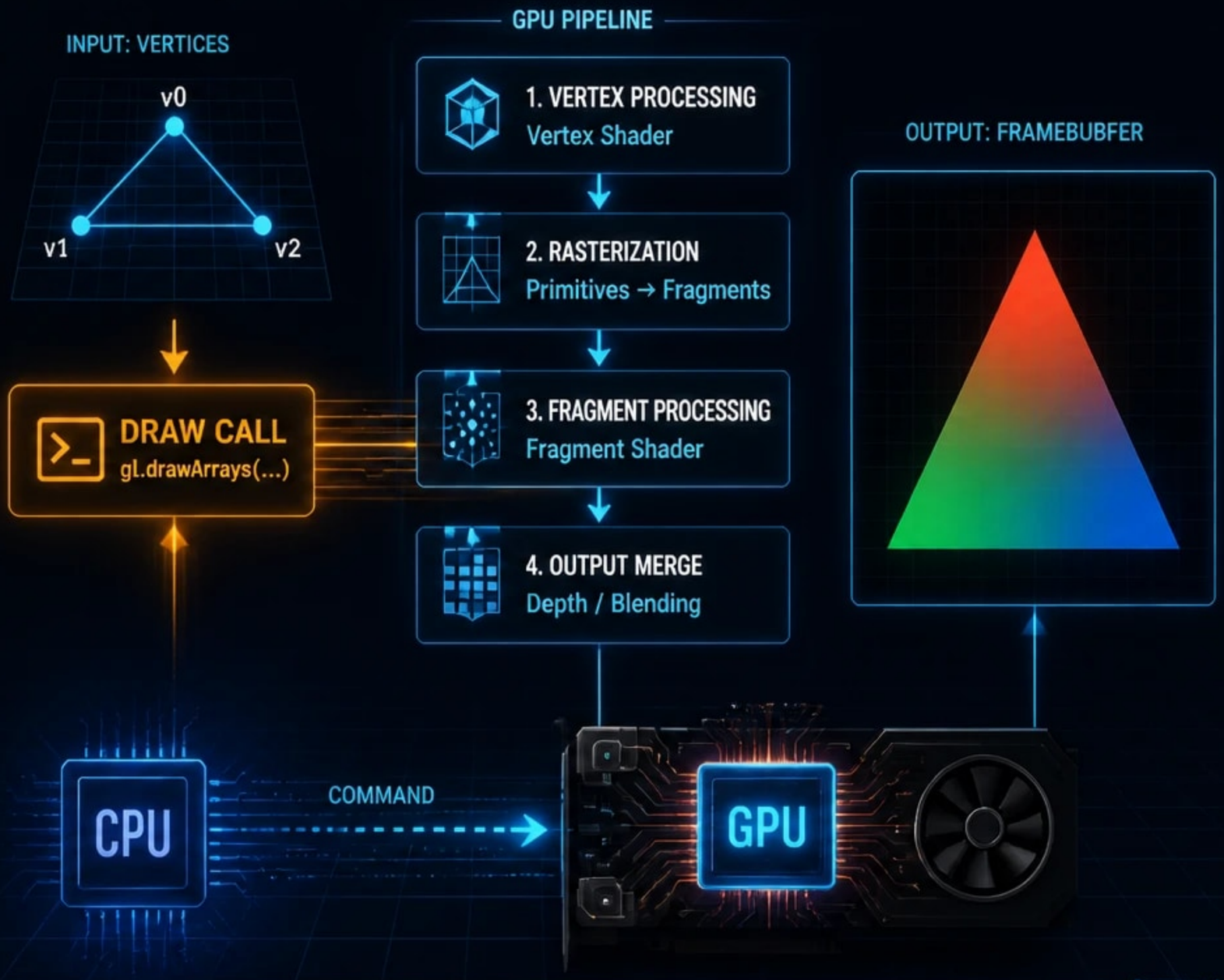
Contoh:

```
gl.drawArrays(
  gl.TRIANGLES,
  0,
  3
);
```

Makna:

- mode = TRIANGLES
- first = 0
- count = 3

Draw call memicu pipeline.



Pipeline WebGL Fundamental

1 JavaScript




```
const gl =
canvas.getContext
('webgl2');

gl.drawArrays(...);
```

Aplikasi Web mengontrol pipeline melalui JavaScript

2 Vertex Data



Kumpulan data vertex: posisi, normal, warna, uv, dst.

3 Buffer



Data disimpan di GPU buffer (VBO)

4 Attribute

- a_position
- a_normal
- a_uv
- a_color
- ....

Buffer di-interpretasi sebagai atribut vertex (layout)

5 Vertex Shader



VS

Setiap vertex diproses untuk transformasi ke clip space

6 Primitive



Vertex dirangkai menjadi primitive: POINTS, LINES, TRIANGLES, dst.

7 Rasterization



Primitive diubah menjadi fragment (piksel kandidat)

8 Fragment Shader



FS

Setiap fragment menghitung warna, lighting, tekstur, dsbt.

9 Framebuffer



Hasil fragment ditulis ke Framebuffer (Color, Depth, dst.)

10 Canvas



Hasil akhir tampil di HTML Canvas pada browser



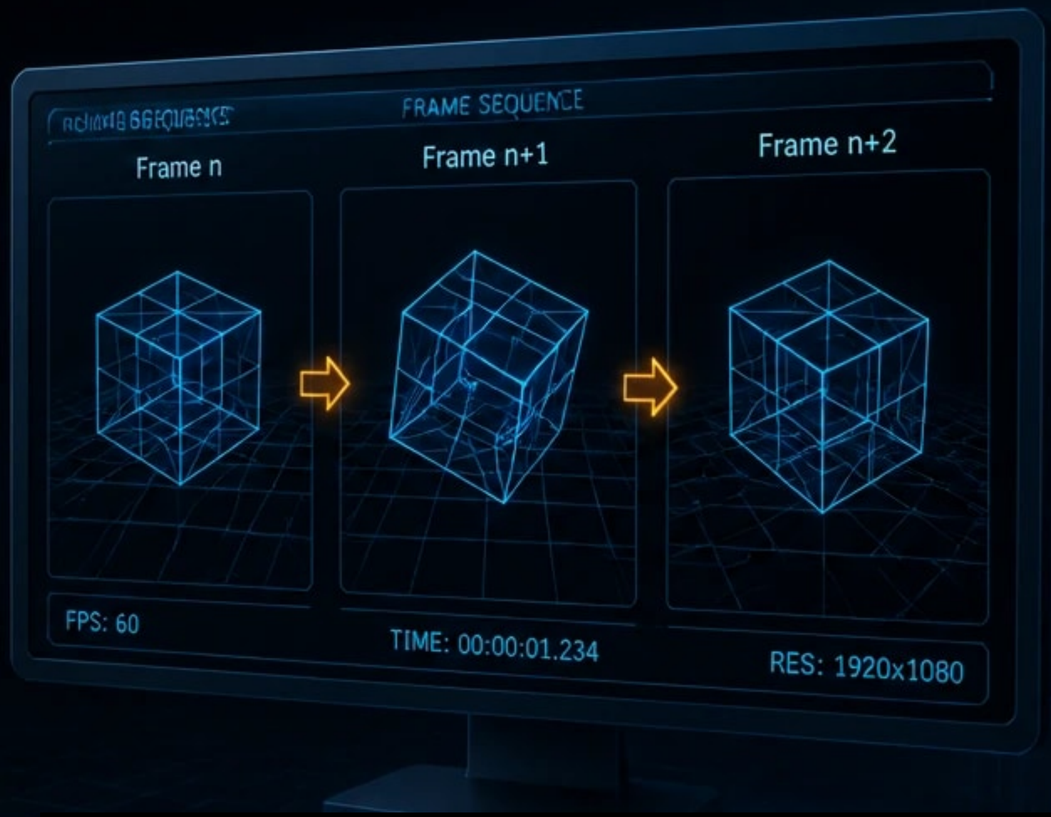
Grafika Komputer

Rendering Loop

Untuk animasi:

```
function render() {
  update();
  draw();

  requestAnimationFrame(render);
}
render();
```



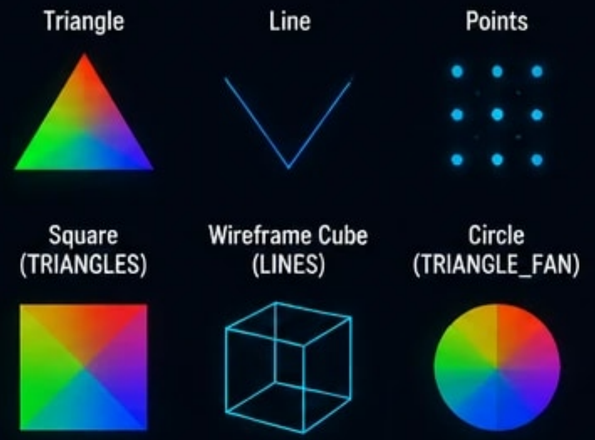
Pola:

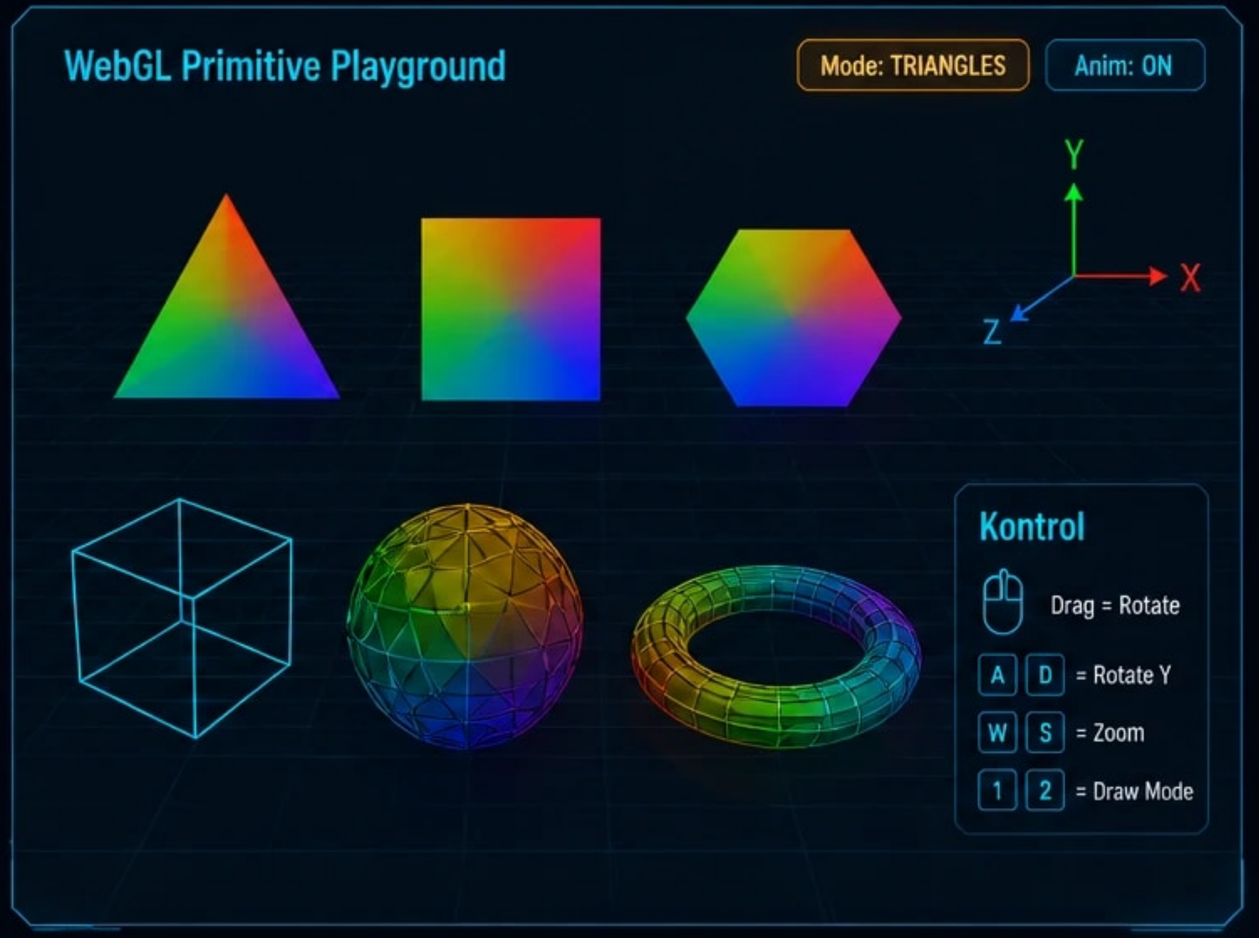


Praktikum: WebGL Primitive Playground

Aplikasi menampilkan:

- *triangle*
- beberapa primitive
- *vertex color*
- minimal dua *draw mode*
- animasi sederhana
- interaksi *mouse* atau *keyboard*





Tujuan:
menghubungkan buffer, GLSL shader, attribute, shader program, dan draw call.



RENCANA DAN OUTPUT PRAKTIKUM

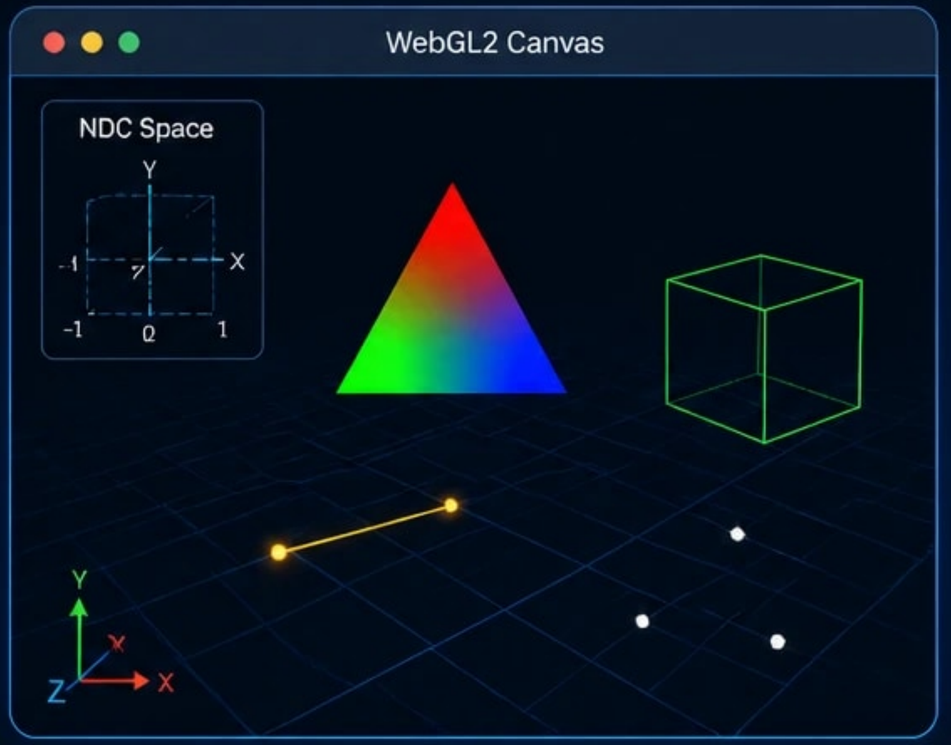
TAHAPAN RINGKAS

1. Create WebGL2 Context
2. Prepare Vertex Data
3. Create Buffer
4. Write GLSL Shader
5. Compile & Link
6. Connect Attribute
7. Draw Primitive
8. Add Color
9. Add Animation
10. Add Interaction



OUTPUT MINIMUM

- ≥ 3 primitive (triangle, line, point)
- ≥ 2 draw mode (contoh: TRIANGLES, LINES)
- vertex color (per-vertex color)
- 1 objek bergerak (animasi posisi/rotasi)
- 1 interaksi (mouse / keyboard)



VERTEX SHADER (GLSL)

```
#version 300 es
precision highp float;

in vec3 aPosition;
in vec3 aColor;
out vec3 vColor;

void main() {
  gl_Position = vec4(aPosition, 1.0);
  vColor = AColor;
}
```

FRAGMENT SHADER (GLSL)

```
#version 300 es
precision highp float;

in vec3 vColor;
out vec4 outColor;

void main() {
  outColor = vec4(vColor, 1.0);
}
```